

Received September 3, 2019, accepted September 18, 2019, date of publication October 7, 2019, date of current version October 31, 2019.

Digital Object Identifier 10.1109/ACCESS.2019.2945930

Emerging Trends, Techniques and Open Issues of Containerization: A Review

JUNZO WATADA^{1,2}, (Senior Member, IEEE), ARUNAVA ROY^{3,4}, RUTURAJ KADIKAR⁵,
HOANG PHAM⁶, (Fellow, IEEE), AND BING XU⁷

¹Research Institute of Quantitative Economics, Zhejiang Gongshang University, Hangzhou 310018, China

²Department of CIS, Universiti Teknologi Petronas, Seri Iskandar 32610, Malaysia

³Department of CIS, Universiti Teknologi Petronas, Seri Iskandar 32610, Malaysia

⁴Department of CSE, SRM IST, Chennai 603203, India

⁵InfraCloud Technologies Pvt., Ltd., Pune 411021, India

⁶Department of ISE, Rutgers, The State University of New Jersey, NJ 08855-8018, USA

⁷Research Institute of Quantitative Economics, Zhejiang Gongshang University, Hangzhou 310018, China

Corresponding author: Junzo Watada (watada@waseda.jp)

ABSTRACT Containerization is revolutionizing the way that many industries operate, provisioning major impact to modern computing technologies because it is extra lightweight, highly portable, energy, resource and storage efficient, cost-effective, performance efficient, and extremely quick during boot up. These often facilitate efficient load balancing, low-level system maintenance, server consolidation (for efficient energy and resource utilization) and replication of instances over geographical locations for better fault tolerance to escalate application reliability. However, some recent literature have addressed various challenges (such as *complex networking*, *persistent storage* facilities, *cross data* centers and *multicloud* supports, *security* issues, and *lack* of available, capable container management APIs, etc.) regarding successful container adoption in industries, which might have resulted in a seemingly meager increase in industrial deployments of containerization over the past few years despite bestowing efficient lightweight virtualization. Moreover, a comprehensive overview of containerizations along with their popularity dynamics has still not been found in contemporary literature, which further extends knowledge gap between developers and available technologies. Hence, current study touches upon different technicalities involved in containerization with potential problems and possible solutions along with various important industrial applications to manifest its existing supports and technical hardships. Finally, we have conducted a comprehensive experimental study to compare the performance of VMs, containers and unikernels in terms of CPU utilization, memory footprints, network bandwidth, execution time and technological maturity using standard benchmarks and observed containers to deliver satisfactory performance in almost all aspects, however, are still not free from issues regarding isolation & security, performance stability, lack of available efficient tools for cross-platform support and persistent storage. Unikernels deliver good performance with VM-like isolation but still need to achieve desired technical maturity (in terms of microprocessor stability, process containment, persistent storage, etc.). VMs, on the other hand, are found to provide stable performance throughout, though bigger memory footprints and slower spin up/down remain their biggest weaknesses.

INDEX TERMS Virtualization, containerization, management and orchestration, isolation and security.

I. INTRODUCTION

Virtualization (a method for sharing physical resources among different users and applications) is considered as one of the pioneering concepts initiating modern clouds [1]. Virtualization can help data center resource (hardware, software, energy and money) optimization by *consolidating*

many underutilized machines to a single system. It also helps in process *reliability escalation* and *fault tolerance* by *performance isolation*, process *migration* and *replication* over geographical locations [2].

Conventional virtualization [3] can broadly be of two types: full and para virtualizations. Full virtualization is the creation of virtual versions of the processor, memory, physical servers, storage devices or network resources and I/O devices to run multiple guest OSs on a single physical

The associate editor coordinating the review of this manuscript and approving it for publication was Muhamamd Aleem.

platform such that the resources and the guest OSs are unaware that they are residing in a virtualized environment [4]. the OSs think it is running on bare metal and does exactly same that it would have done on a bare metal processor like try to execute certain privileged instructions without having right privileges. In full virtualization, the central idea is to run unchanged binaries of the OSs as user-level processes on the top of the hypervisor. Since the OS code is unchanged, it does not know that it does not have required privileges for executing certain specific instructions. For example, some privileged instructions that require OS to run in privileged or kernel mode on bare metal for their executions, will create a trap that goes into the hypervisor, which will then emulate the intended OS-functionality. This is known as the trap and emulate strategy of full virtualization. This strategy has few problems such as in some architectures, some privileged instructions might fail silently. In order to solve this problem, in fully virtualized system, the hypervisor will resort to binary translation strategy to know the set of instructions that might fail silently in the architecture and ensure that the instructions are carefully dealt with using binary editing strategy. Full virtualization is employed in VMWare. In another approach to conventional virtualization, called paravirtualizations, guest applications are executed in isolated domains and can be identified as guests and issue commands directly to the host OS to fulfill necessary resource requirements. It requires the modification of source codes of the guest OSs for avoiding problematic instructions, including optimization, letting the guest OS to see the real hardware resources underneath the hypervisor, accessing real hardware resources, employing tricks like page coloring and exploiting the characteristics of the underlying hardware. Zen family uses paravirtualization. As can be seen that the conventional virtualization comes at a cost of high resource overheads [3].

Another approach to virtualization, called container-based virtualization (also known as OS-virtualization, containerization) can be considered as an alternative to conventional virtualization as it potentially reduces resource overhead and thus improves the utilization of datacenters [3]. In containerization, an individual instance (called a container), runs on top of a shared OS kernel (with necessary *isolation*). A key difference between containerization and hypervisor-based virtualization (using VMs) can be noted; as in containerization, the virtualized objects are mainly limited to global kernel resources, which allows containerization to run multiple virtual environments on top of a *common* host kernel of an utilizes fewer CPU, memory and networking resources to provide efficient, scalable and cost-effective resource management solutions in cloud infrastructures, which are the main reasons behind its increasing use. More specifically, container management is best done explicitly by a distributed system, rather than simply leaving it to the user to invoke a container from within each task [5].

Although the recent advancements of virtualization and cloud technologies point to huge number of successful adoptions of containerization in various key industrial

applications, the challenges like lack of cross platform supports and container portability limitations, difficult data management and networking during large scale container deployments, unavailability of persistent storage and multi-cloud supports, lack of efficient tools for container management and orchestration must be addressed with proper care to continue the increasing momentum of containerization in industrial usages in foreseeable future. Moreover, the container *isolation* mechanism is much weaker compared to that of VM, which also contributes to limiting their widespread industrial applications, since enterprise security people cannot implement various-and often fine-grained-security policies that they are required to abide by Hence, escalation of security and isolation strategies becomes another prime need to continue the increasing momentum of containerization in industrial productions.

Porting containers from one OS family (say Linux) to another (say Windows) is more complex than the porting of VMs. Additionally, a containerized Linux application needs Linux host, which is not necessary for VMs. Also, it is quite hard to get container support for macOS as traditional containerization mainly supports Linux and Windows platforms. Hence, there is no native way available to deploy Dockerized applications on macOS except the relatively costly technique of employing Linux-based VM as middleman between macOS and Dockerized application, which may increase networking conundrum during large scale deployments. This clearly make containers less portable than VMs because of the unavailability of necessary cross-platform supports. Without persistent storage, creating and deleting a large number of containers with workloads, including thousands of tasks in a distributed environment, is often costly significantly high system workloads [6]. This workload can be reduced by sharing containers across multiple tasks at the cost of losing isolation, which increases the chance of a security breach [6]. This sharing of jobs among a large number of containers is managed by different orchestration tools and cluster managers such as Google Borg, Docker Swarm Manager, and Kubernetes [7]. A number of security vulnerabilities are found in many official and community container images (considering all versions) as they have not been regularly updated [8]. These vulnerabilities are mainly propagated from parent image to child and can cause severe security breaches. In the absence of automated and systematic methods of applying security updates, containerization might face opposition in industrial usage. Hence, to maintain the current thrust of industrial usages of containerizations in foreseeable future, there is an urgent need to analyze containerization benefits and weaknesses along with several underlying technicalities. Following are the contributions of the present work:

- ① Performed a detailed study of recent advances of various aspects of containerization such as, container networking; container security & isolation; container cloud and their applications; management & orchestration; and performance analysis.

② Discussed various contemporary concepts such as the differences between hypervisor-based virtualization and containerizations, Linux kernel supports for containers (cgroups and namespace), application vs machine containers, different key enabling technologies (such as container management and orchestration, container networking, security & isolation, different advanced Linux security modules) and key industrial container applications.

③ Discussed different research challenges and future trends of lightweight virtualization.

④ Performed experiments to compare several aspects of containerization with that of VMs and unikernels.

Before going to the detailed study, it would be apt to briefly touch upon the outline of the current work that include research background & recent literature (Section II), difference from hardware-based virtualization (Section III and Section IV), different key enabling container technologies (in Section V), container security and isolation (in Section VI), key container applications in industries (Section VII), research challenges and future trends (in Section VIII), a number of real-time experiments are conducted with well-known benchmarks (Section IX) for showing near real-time performance, easy software deployment, security & isolation of modern containerization. Finally, the key findings are presented in the conclusion (Section X).

II. RECENT RELATED LITERATURE

Over the past few years, with the increasing popularity of containerization, considerable been conducted on aspects of containerization that can be broadly classified into the following categories.

A. CONTAINER NETWORKING, MIGRATION, CONTAINER CLOUDS AND THEIR APPLICATIONS

Container-based clouds have recently gained momentum in various industrial applications, although they must deal with several complex challenges regarding networking and configuration from both customers and providers, when deployed in an existing cloud system [12]. Thus, providers often need to install additional proxy servers and Network Address Translation (NAT), while customers need to use subdomain names and randomly assigned port numbers to access specific services [12]. Moreover, containers running within a single VM (provides better CPU utilization [14]) are often addressed by specific subdomain names instead of a public IP, and applications on containers are assigned with random TCP/UDP ports rather than the regular ports used by the applications. Although this strategy enables containers to function like network servers, it has several limitations, which makes configuration and management of container-based clouds relatively difficult and confusing. To reduce this confusion, Kim *et al.* [12] suggested assigning public IP addresses and port number to application running in a container, where the applications can open TCP/UDP ports regardless of other tenants residing on the same host. However, the difficulties escalate with the application size,

where network topology and routing algorithms are two key parameters for communication performance evaluation [13]. Network topology becomes fixed after application deployment and might force static/dynamic routing protocols to penalize performance. Hence, the developers need to understand the optimal network configuration for distinct communication patterns apriori and FlexTuner [13] is one such tool that allows developers to analyze communication cost of their application and find a suitable network topology along with routing algorithm. Although this study did not consider all available network topologies, it can still be successfully used on a limited scope with some fundamental network topologies. Network performance (throughput) of hypervisor based virtualization (preferably, Xen) is often degrades because of additional complexities during transmitting and receiving packets on long data transmission paths between the guest and hypervisor [9]. In contrast, Varma *et al.* [14] reported that an increase in the number of containers (or cluster nodes) sharing the same CPU (or in a VM) may have a negative impact on network I/O throughput due to the increase in context switch latency (latency in storing/restoring the state of a process, for halting/resuming any execution at/from point of time), thereby creating a bottleneck in big data environments. IBM mitigated networking issue such as scarcity of IP addresses while building their container cloud service, by using shared routers between multiple tenants [17].

Docker is considered an excellent tool for wrapping existing single-tenant applications and run them as multitenant applications, but the question remains as to how to migrate legacy applications and run them as multicloud services using Docker containers. Skysport uses containers to deploy and execute scientific workflows in a cloud platform but is not designed to be a cloud service. Hence, multitenancy, security and isolation are not of its prime requirements. Additionally, Skyport does not discuss various issues regarding migration of legacy applications to the cloud. Hence, there is a need to develop legacy application migration mechanisms and a reusable architectural pattern for migrating legacy web applications to multitenant, elastic and highly available cloud service using Docker and Cloudant persistence layers was presented in the work [11], which shows the applicability of containerization in various legacy code migration applications [11].

Workflows are widely used abstraction for representing large scientific applications and executing them on distributed systems such as clusters, clouds, and grids. Several tasks within a workflow might need different environments for execution, which remain mostly unknown apriori, and hence, during migration to another system, migration is highly likely to fail due to differences in the OS, installed applications, available data, and other dependencies [6]. In their work, Zheng *et al.* [6] integrated container technology into an existing workflow system because of the advantage of containerization in providing a well-defined execution environment at the OS-level and analyzed the trade-offs between performance, isolation and consistency. Experimental results show

that the large overhead due to thousands of tasks, launching and destroying numerous containers can be efficiently handled by sharing containers across multiple tasks in the cost of losing precious isolation. Management of containers is best done explicitly by a distributed system with a microservice architecture, rather than a traditional monolithic software structure (all components of the software run as a single process) [41]. In a simple scenario, a monolithic structure has several advantages, such as easy deployment and networking, which, become harder with complex system requirements and high demand. For example, changes in one component can influence seemingly unrelated areas of the application supporting monolithic architecture that increases the possibility of system failures.

Microservice architecture, supported by containerization, can reduce the complexity of monoliths with more flexible distributed systems of lightweight, narrowly focused and independent services (written in different languages and tools), which use RPC, TCP or HTTP to communicate with each other, and update and scale of components. Although microservice architecture can resolve various drawbacks of monolithic systems, and can also introduce some new challenges, such as the “*service discovery*” problem, “*complex deployment*” and “*log aggregation*” [41]. Deployment and log aggregation can be handled with the necessary expertise and available cluster manager tools such as Google’s Borg System and Kubernetes, which can run hundreds of thousands of jobs from many thousands of different applications, across a number of clusters each up to tens of thousands of machines [7]. In contrast, service discovery problems often become much harder to solve. The problems occur because service over a complex networking layer cannot be counted on to be present at a given moment, and hence, the consumer service needs a real-time mechanism to locate the provider service. A number of service discovery mechanisms have been designed to address this issue, of which, Consul [41] bundles service discovery with strongly consistent key-value stores, robust monitoring and health checking. Consul relies on a central set of agents running in server mode in each data center to maintain the cluster state and stops working unless the central agent is unable to form a quorum. Consul’s raft-based [44] quorum of server nodes can provide strong consistency. Synapse [41] is another tool for service discovery and management that relies on Docker, AWS EC2 and Zookeeper. CoreOS [41] provides a service directory as a basis of their OS through a distributed, consistent key-value store or shared configuration and service discovery service, called *etcd* [41].

SaaS providers are faced with the increasingly relevant but nontrivial question of whether the container technology can play a vital architectural role in the development and operation of multitenant SaaS applications [27]. The popular shared-everything multitenant architecture is more cost-efficient than containers for much weaker security isolation between tenants. Multitenant SaaS applications typically rely on complex application-level middleware services

for performance isolation [45], tenant data management deployments [47]. In principle, it is possible to simplify these middleware services by shifting some functionality to a container orchestration layer. However, container orchestration middleware has some hard pen challenges such as unified configuration management and automated management of service dependencies [48]. In summary, there does not exist a simple answer to whether container technology supports a strengthening or weakening of multitenant SaaS architectures. Truyen *et al.* [27] presented an in-depth analysis of Strengths, Weaknesses, Opportunities, and Threats (SWOT) of a container-based architecture for multi-tenant SaaS applications, where the strengths are because of Resilience (automated volume migration from failed PoD to a newly started PoD have significantly reduced data synchronization time), Elasticity (amount of PoDs behind service can easily be scaled), Dynamic Reconfiguration (up and downgrading a service to a newer version becomes simple), Cloud Portability (supporting examples are, Docker, which can run on any Linux-family OS; Kubernetes is offered as a service by several cloud providers, etc.); Weakness is mainly because of poor security and isolation; Opportunities are due to configurable security isolation between tenants and simplified performance isolation; and Threats are due to increased management complexity. Other container cloud applications include cloud architecture for real-time full motion video target tracking [20], and distributed systems of microservices using Docker containers [41]. Different technical hardships, involved in container networking, are covered in detail later.

B. CONTAINER ISOLATION & SECURITY

Isolation & security are often considered prime requirements in containerization, otherwise a compromised container might affect other colocated container instances on the same host [5], [16], [17]. Moreover, by subverting the Linux kernel, adversaries can take over all containers residing on the same host (known as a single point of failure). Container *isolation* enables protection of container-specific information from unwanted leakage for maintaining user-privacy (see Section VI). Soltesz *et al.* [15] provided insights into *resource*, *security* and *fault* isolations for avoiding crosstalk, unwanted snooping and fault propagation between containerized systems, although container usage for provisioning security isolation may not seem propitious [1], [16]. Xavier *et al.* [9] also found container inability to provide the desired *isolation* during the execution of high disk-intensive workloads and proposed the consolidation of different workloads as a possible solution. Gerlach *et al.* [10] used Docker for *isolation* to mitigate issues relating resource utilization, research method reusability, software deployment, software version, and software compatibility, which could escalate difficulties while reproducing computing environments (to obtain identical results). In another work regarding reusable architecture for migrating legacy web applications to clouds, Slominski *et al.* [11] designed a workflow service

architecture that supports multiple tenants consuming minimal resources. Tenant isolation was implemented using separate databases protected with tenant-specific credentials.

In general, container protection methods are broadly classified into two categories called userspace isolation or host-based intrusion detection and kernel security hardening mechanisms (using `Seccomp`, `AppArmor`, `SELinux`). However, applying both mechanisms to container environments is not straightforward for several reasons. First, there are limitations in properly deploying them in container environments where part of the workload is executed on the host and part inside the container, in which case multiple processes and applications should be grouped and protected together. Second, their practical application to noisy container environments is not straightforward. `Docker`'s security relies on three components called: (i) process isolation at the userspace managed by the `Docker` daemon; (ii) enforcement of this isolation by a kernel; and (iii) network operation security. The `Docker` container relies on `Linux` kernel features, including namespace (enabled, by default) and, `cgroup` (not by default, but must be enabled on a per-container basis). The `Docker` file-system leverages the copy-on-write mechanism, and all containers can share a basic file system image. Writing data to the file-system and, guiding it to write to the associated file, this mechanism can not only significantly reduce the consumption of resources in modifying the data of the container, but also effectively ensure data isolation between containers [42].

Default `Linux` isolation is relatively strict; however, it is not secured in front of ARP poisoning as colocated container instances share the same network bridge Combe *et al.* [16]. The kernel enforces this security mechanism along with with kernel security modules such as `Seccomp`, `Apparmor` and `SELinux`. `Docker` uses network resources for image distribution, controlling the `Docker` daemon, downloading images from remote repository that are verified with a hash and a connection to the registry made over `TLS` [16], where content trust relies on the update framework. In an interesting study, Shu *et al.* [8] have investigated the state of security of 356,218 images downloaded from both official and community `Docker` Hub images and concluded with three important points: (i) both official and community images contain 180 vulnerabilities, while considering all versions; (ii) many of the images have not been updated for approximately 100 days; and (iii) vulnerabilities generally propagate from parent to child images. Hence, there is a strong need for more automated and systematic methods for applying security updates to `Docker` images.

C. CONTAINER MANAGEMENT & ORCHESTRATION

Cloud and data center employ orchestration frameworks to manage resources (at software and hardware layer) required by the applications; deploy a set of virtualized services in order quality objectives of both users and service providers [49]. Orchestration frameworks manage resource allocation of VMs and containers by exploiting the resource

provisioning parameters provided by underlying platforms (hypervisors and OS). Compared to hypervisors, containers can provide both physical (CPU, memory, etc.) and OS-level (CPU scheduling, swapping, etc.) resources [50]. Moreover, containers possess multiple options to allocate a specific resource (e.g. CPU-shares and CPU-sets [50]). The key feature of management frameworks is to launch applications with the lowest possible latency. Apart from this, management and orchestration frameworks must provide policies for application placement (placing applications on the physical hosts with available required resources) providing the best suitable consolidation [50]. Li *et al.* [18] identified the necessity of a flexible and efficient framework for resource management in a container-based cloud environment and designed a REST service framework named the Resource-Oriented Network (RON) to decouple resource *representation*, *control* and *data* planes offering fine-grained resources for processes, tasks, containers, clusters, etc., by encapsulating `Linux` resource control models. Access to RON was secured by employing the RBAC [51] policy. Later, Baresi *et al.* [52] incorporated a MAPE (Monitor, Analyze, Plan, Execute)-based approach to allocating and auto-scaling CPU cores to running applications in containers as well as allocating a fixed amount of memory per core using a discrete-time feedback controller. Furthermore, Baresi *et al.* [19] presented a `MicroCloud` architecture that can provide fine-grained, dynamically adaptive, multi-level and topology-aware resource allocation to multiple applications running inside the containers. Other related noteworthy literature is listed in Table 1.

D. PERFORMANCE ANALYSIS AND OTHER EXPERIMENTAL SETUPS

Hypervisor-based virtualization is mostly avoided in creating traditional high performance computing clusters as they often cannot attain the same performance obtained in non-virtualized setups due to their inherent performance overhead [9], [59], whereas containerization [15] is considered to be a more effective alternative that might provide near real-time performance in terms of cost-management, performance isolation, checkpoint and live-migration. To corroborate, in [9], the authors conducted experiments to effectively compare current container technologies both in terms of performance and manageability, while running on `MapReduce` clusters and found that all container technologies can provide near real-time performance. However, among them, `LXC` was found to offer the best relation between performance and management capabilities (especially performance isolation). People might argue about the performance of `LXC`, but still, the overall result looks corroborating to enriching the popularity of the containerization, especially, for `MapReduce` clusters. Again, containerization is better suited for the scenarios, where the requirement is system virtualization with a high degree of efficiency and good isolation; examples include HPC clusters, grid, hosting centers, and `PlanetLab` [15]. VMs can provide stronger isolation but

TABLE 1. Studies with domains covered, used technologies (marked with “✓”), and major findings.

Works	Security		Performance		Network		Orchestration		Technologies	Summary of the findings
	Isolation	Safety	Devices	Apps.	Internal	External	Provisioning	Management		
[9]	✓		✓		✓				✱ LXC, OpenVZ, VServer	✱ Performance comparisons of current containerizations for MapReduce clusters; ✱ tested performance isolation for current containers; ✱ LXC offers best relation between performance & isolation.
[10]	✓			✓			✓		✱ Docker, AWS/Shock	✱ Addressed issues pertaining to resource utilization during software deployments and reproduction of computation environments; ✱ developed Skyport (extension to AWS/Shock) can reduce complexities regarding resource utilization during software deployments.
[11]	✓			✓					✱ Docker, IBM Bluemix Workflow Service (β)	✱ Developed reusable architectural pattern to migrating legacy web applications to cloud service; ✱ Used Docker and Cloudant; persistence layers for delivering multi-tenant cloud service by reusing legacy codebase.
[12]					✓	✓			✱ dorCloud, AWS EC2, SDN	✱ Assigned unique identifiers to the containers in a VM and forwarded their traffic using SDN; ✱ proposed net architecture to perform M-to-N mapping between net address and containers.
[13]						✓			✱ Mininet, POX, Linux Containers	✱ Designed flexible container-based tuning system for analyzing communication cost among containers and finding suitable network topologies; ✱ used SDN to connect and direct network traffics among containers.
[14]			✓		✓				✱ VMware ES1, Hadoop v2.7.0, Hive v0.12, Docker	✱ Analyzed network performance by increasing number of container on a physical host; ✱ observed a bottleneck, where Hadoop cluster throughput is not-linear with the number of nodes sharing same CPU. This is due to virtual network layer adds significant delay to RTT of data packets.
[15]	✓		✓						✱ Linux VServer, Xen	✱ Provided insights into resource, security and fault isolation to avoid crosstalk, illegitimate snooping and fault propagation; ✱ it contrasts Linux-VServer architecture with current generations Xen.
[1]	✓		✓		✓				LXC, Docker, Kubernetes	Presented detailed analysis on various containerization techniques.
[16]	✓		✓		✓				✱ Docker	Pointed various vulnerabilities in a complex container ecosystem as insecure local config; weak local access control; vulnerabilities in the image.
[17]	✓				✓				✱ IBM Containers	✱ Discussed about various real-time architectural challenges, security conditions and services capabilities; ✱ Developed a cloud container service considering security and isolation on a shared infrastructure.
[5]	✓								✱ Docker	✱ Targeted security policy management by constraining resources; ✱ mitigates poor isolation issues by enforcing policy checks on Docker container management operations and allows administrators implementing principles of least privilege.
[18]	✓		✓		✓		✓		✱ Docker	✱ Two trends towards flexible efficient cloud computing paradigm: Resource-as-a-Service (RaaS) and container clouds; ✱ developed a flexible and efficient framework for container resource management using RBAC; ✱ presented a RBAC architecture to secure access to RaaS in cloud decoupling resource representation, control and data planes.
[19]			✓				✓		✱ Docker, AWS EC2, Vagrant	✱ Adopted MAPE technique to allocate CPU cores and a fixed amount of memory per core to the running applications in containers; ✱ Micro-Cloud provides multiple containerized applications with fine grained resource allocations; ✱ Micro-Cloud uses EcoWare framework for self-adaptive cloud applications.
[20]			✓						✱ OpenVZ, Xen	Tracked full motion videos by parallelly processing video frames on multiple containers to optimize computing resources.
[21]									✱ Micro-Container	✱ Proposed micro-service architecture for deploying web applications using containers; ✱ developed service micro-container for addressing resource scalability by reducing memory consumption and response time.
[6]			✓				✓		✱ Docker	✱ Created isolated, portable and self-contained environment to deploy scientific workflow applications using containers; ✱ it considers way for integrating container technology into workflow systems.
[22]			✓						✱ LXC, Ceph	Analyzed several containerization features that if leveraged correctly can reduce complexity of reproducing experiments.
[23]	✓		✓		✓		✓		✱ AWS EC2, Docker	Leveraging and evaluating containerization environment for enabling assembly, provisioning and execution of dynamically tailored instances for satisfying application specific service middleware communication requirements.
[24]			✓		✓				✱ kvm, Docker, LXC, OSv	Analyzed hypervisor and containers using benchmarking tools like ✱ NETBENCH for CPU, FPU and memory system performance; ✱ noLoop for CPU benchmarking; ✱ Linpack benchmark for testing system performance with simple linear algebra problem.
[25]			✓		✓				✱ Docker, OVS, Linux Bridge	✱ Bonnie++ for checking disk performance.
[7]	✓			✓			✓	✓	✱ Google Borg, Kubernetes, Linux Containers	Performance comparison between of master-slave and nested container implementation of microservice architectures.
[8]		✓							✱ Docker, Docker Hub	Detailed comparison of Google Borg (architecture, scheduling, security isolation, performance isolation, resource reclamation, availability, utilization) and how it makes Kubernetes more sophisticated.
[26]			✓						OpenVZ, Xen, XenServer	✱ Possible reasons of Docker vulnerabilities; ✱ official and community images have vulnerabilities; ✱ many images are not updated regularly; ✱ vulnerabilities are propagated from parent to child images.
[27]				✓					✱ Kubernetes	Performance analysis of OpenVZ, Xen and XenServer.
[28]						✓			✱ LXC, Multipath-TCP	SaaS are hosted on top of scalable cloud infrastructure. Model strengths: Resilience: failure of a node can be recovered with min data loss; Scalability: performance remains when usage load increases; Elasticity: New loads are added dynamically added based on user load. Weakness: poor security & isolation. Opportunities: configurable security & isolation among tenants; simplified performance isolation. Threats: Hybrid virtualization, increased management complexities.
[29]				✓					✱ Docker	Provided an insight on migrating container over WAN using Multipath TCP to avoid network congestion.
[30]	✓	✓							✱ Docker	Docker container based resource allocation framework for minimizing application deployment cost in DC and for supporting automatic scaling while cloud application workload varies.
[31]				✓					✱ Docker, OpenStack	Categorized the threats caused by container and devised a secured framework for Linux containers to detect these threats.
[32]				✓					✱ Docker, OpenStack	Compared High Performance computing applications on Docker and VM.
[33]			✓						✱ kvm, Xen, Docker	Hard to use containers to manage cloud without sacrificing many of containerization benefits. Challenges for improving operation efficiency; ✱ minimize cross-configuration of services; ✱ maintain running services states; ✱ provide safe access to host resources.
[34]		✓		✓		✓		✓	✱ Linux Containers	Carried out I/O & network performance analysis on Docker & hypervisors.
[35]			✓						✱ Docker	✱ Provides the set of capabilities required for container orchestration platform embodying design principles; ✱ provides non-exhaustive suggestions for identifying and implementing key mechanisms for container orchestration.
[36]				✓			✓	✓	✱ Amazon ECS	✱ Addressed the bottlenecks in container management system that affects container scalability; ✱ this study shows that core scalability has bottlenecks at virtualization layer for storage and network devices and that for the container bottlenecks at various components inquiring mount points.
[37]							✓		✱ Flannel, Calico, Swarm Overlay	Introduced a framework for intelligent scheduling, scalability, resource efficiency with joint optimization of containers and resources.
[38]							✓		✱ Docker, Xen	✱ Provided comparative performance analysis of various existing container network solutions; ✱ their experiments shows Calico's superior TCP throughput, which may be situation specific.
[39]		✓							✱ Docker	✱ Compared the performance of VM based and container based VNFs.
[40]							✓		✱ Linux Containers, Docker	✱ Introduces an additional security by enabling and extension of specific SELinux to Dockerfile format; ✱ Docker uses simple Dockerfile format, which defines necessary actions for generating Docker image for instantiating containers.
[41]				✓				✓	✱ Docker	✱ Addresses the issues like lack of cognitive and adaptive models for automatic container orchestration and analyzed state of art solution.
[42]	✓	✓							✱ Docker	✱ Proposes a method called Serfnode, fully decentralized automated open source solution for service discovery problem within a micro-service cluster; ✱ Serfnode is a non-intrusive Docker image that comprises one or more Docker containers; ✱ New image can be put to a Serfnode-cluster, where it provides service discovery mechanism, monitoring and self-healing.
[43]			✓	✓					✱ Linux Container OpenSAP, PaceMaker	✱ Addresses kernel vulnerabilities due to Docker escape attack and proposes a detection technique by inspecting namespaces.
										✱ Presented an approach for high Availability (HA) of cloud applications using application design, placement and recovery policies; ✱ a middleware with a set of HA agents is used to compensate limitations of Linux containers in achieving HA.

TABLE 2. Conceptual differences between VMs and containers.

	VMs	Containers
Conceptual dissimilarities	Not process-specific; needs separate OS to run processes.	Process-specific; needs necessary OS components for running processes.
Abstraction	Hardware-level abstraction, needs share in CPU cores, RAM, cache and storage [53].	OS-level abstraction; needs shared bins, libs and executables [54].
Standardization	Host/guest utilizes standardized system images functioning analogous to bare-metal computers [55].	System environments are less standardized in OS- and kernel-specific areas with varying degrees of complexity [55].
No. of kernels	Multiple kernels for multiple VMs on a PM.	One kernel for different containers on a PM.
Boot Process	Initiated using standard boot process that results in initiation of a number of hypervisor processes on the host.	Containerized applications are initialized directly or employing a container-aware initiation daemon, e.g. <i>systemd</i> , which is equivalent to normal processes on host.
Boot time	It takes few minutes for booting	It takes few seconds for booting
Security isolation	Utilizes internal OS objects (like UIDs, PIDs, Sys-V Shm, etc.) by separating namespaces (contextualization) as well as access controls (filters) for security isolation [55].	Utilizes contextualization and filters at hardware abstraction layer (memory address space, PCI address, device instructions etc.) for security isolation.
Live-Migration time	Takes more time in highly write-intensive environments, during long distance <i>live-migration</i> of VMs running heavy applications.	Relatively, shorter due to its small size. Instead of migrating whole OS (for VM), it migrates specific OS components along with its process under execution.
Host/guest Architecture	Running guest kernels can differ from host, along with, more limited access to host's memory and storage, management.	Executes host kernels at guest by varying package tree or distribution in such a way that container kernel operates similar to host.
Live-Migration Predictability	Not satisfactory [56] though <i>post-copy</i> and <i>hybrid</i> offer better predictabilities [57], [58].	Predictability often better than VM due to its relatively smaller memory footprint.
Checkpoint & Resume	Possible [17].	Possible [17].
Administrative Power	Root [17].	Root [17].
Portability w.r.t Backends	VMs are more portable than containers; VMs are OS and processor (x86, x64, ARM, etc.) independent.	Containers offer lower portability than VMs; Containers can be executed on host OS; similar to guest, or its other distribution (mainly Linux)
Performance characteristics	Relatively high startup time, storage and performance overheads is required for VMs.	Containers provide negligible startup time, storage and performance overheads in comparison to hypervisors [9], [24], [59]
Live System Update	Very hard to implement, to the best of our knowledge [17].	Yes [17].

are also less efficient than containers. Hence, containerization might often be found to be a better and more accepted alternative.

Container networks play very important role in the container life-cycle system [37]. The works [36] and [37] have investigated the current development of mainstream container models and network solutions such as, Flannel, Docker Swarm Overlay and Calico. They found that Calico (BGP based implementation using three routing layers) has a complicated configuration but shows high performance and very similar TCP throughput as the host because Calico uses data center network structure during network deployment; hence, there is no need to rely on independent network equipment, which makes transfer efficiency higher than its competitors. Calico maintains a separate IP address for individual containers, which makes its routing table relatively large with less mobility support. However, Flannel (network planning service for Kubernetes) has relatively simpler deployment and configuration, but its performance is not satisfactory for various industrial applications because Flannel treats each container with a separate subnet architecture and does not allow fixed IP container drift, which makes it more usable for container clusters with low requirements for flexibility. Docker Swarm Overlay is more suitable for simple network construction because of Overlay's dependence on VxLAN for cross-host communication with encapsulation and decapsulation that penalize with Overlay's poor network performance [37].

Network Function Virtualization (NFV) has been popularized with VM implementation, although there are several shortcomings, such as hardware virtualization, which incurs considerable performance and efficiency costs; heterogeneous packaging and virtualization platforms may result in fragmented distribution and complex orchestration; VM images are often large and require significant time for deployment and stateful migration among hosts; network I/O may

also suffer from various configurations. Some of these problems can be avoided using OS-level virtualization because applications in containers run on the host OS without indirection of underlying hardware, which allows them to run more efficiently than their VM counterparts and allow higher application density on the host; Docker's novel packaging might remove some of the hosting variability requirements for Virtual Network Functions (NFVs); containers do not require packaging OS in their images, which require less disk space and deployment time [38]. Due to the availability of most of the above facilities, Docker becomes a natural fit for NFV. Experimental results of [38] found Docker has lower network latency cost and lower variability than equivalent Xen VMs running on the same software.

III. HYPERVISOR VS CONTAINER-BASED VIRTUALIZATION

Hypervisor-based virtualization and containerization are two most commonly used techniques for server abstraction and efficient resource utilization in data center environments [50]. Hypervisor-based virtualization involves running a hypervisor for sharing virtualized server resources among multiple VMs [1], whereas containerization involves OS resource virtualization using daemon or container engine.

A. HYPERVISOR-BASED VIRTUALIZATION

It is carried out at hardware-level by virtually distributing physical resources (using a software layer, called hypervisor or VMM, placed between OS and hardware) among multiple co-located VMs running parallel. Hypervisors can be of *two* categories, *type-I* (bare metal) and *type-II* (hosted hypervisor). *Type-I* (Xen) is placed directly on top of hardware, whereas, in *type-II*, an underlying OS that runs above hardware (e.g., kvm). Thus, *type-I* performs better as it avoids an additional OS layer, though usage of *type-I* and

type-11 are mainly need driven. Hypervisor-based virtualization [60] offers following (see Table 2):

① *Performance Isolation*: Resource consumption and failure of one VM must not affect other co-located VMs.

② *Server Consolidation*: Supports consolidating many under-utilized servers to single machine for significantly reducing energy usage to 90% (major 5G support) [61].

③ *Live-Migration*: Supports *live-migration* of VMs from one physical host to other for providing better *fault tolerance*, *load balancing*, *resource allocation*, *high availability*, *disaster management*, *failure recovery* and efficient system scaling. Migration Migration might seem redundant for *stateless* containerized applications, but necessary for implementations like databases, message broker, state-coordination services, etc., [15].

⚙️ How containers implement *live-migration*? Containers implement *live-migration* using *Checkpointing* and *Restart* features (implemented as loadable kernel modules along with the set of userspace utilities) such as checkpointing running container state and restart it later on the same or a different physical host, in a way transparent for running applications and network connections [62]. *Checkpointing* is done in *three* stages [62], called *freeze container processes* (move process to previously known state and disable network); *dump container* (collect and save complete state of all container processes as well as container to a *dump* file); *stop container* (*kill* all processes and unmount container *file-system*), whereas *Restart* (on same or other hosts) requires *restart container* (create a container with the same state saved in *dump* file); *restart process* (create all processes within container in frozen state and restore all of their resources from *dump* file); and *resume container* (resume process execution and enable networks and next, container continues with normal execution). Container *live-migration* implemented using checkpointing and restart is described in following steps [62]:

① Transfer container *file-system* to destination server (*file-system* synchronization); ② Freeze all the processes and disable networking; ③ Collect all resources and dump them to a disk file; ④ Since container is still running on source and hence, dirty pages are continuously generated, which motivates the second time container file synchronization; the destination; ⑤ Restart container at the destination; ⑥ Resume container execution at destination; ⑦ Stop container at source; ⑧ Destroy container at source. To the best of our knowledge, Voyager [63] is a *file-system-agnostic* and *vendor-agnostic* consistent *live* container migration service designed following Open Container Initiative (OCI) principles.

④ *Replication*: Supports VM replication over geographic locations as proactive measure of *fault-tolerance*.

Containerizations provide above features with less *memory* and *time* footprints. VMs are often found providing better *security & isolation* due to hardware level partitioning and abstraction, whereas, containers provide OS-level abstractions. Container security can be enhanced using various security mechanisms discussed later.

B. CONTAINERIZATION (⌘): TYPES & SUPPORTS

In hypervisor-based virtualization, each VM runs separate OS, and applications on top (see Fig. 1). On contrary, containerizations (termed as *type-III* or *type-C* hypervisor) virtualizes OS level resources encapsulating standard OS processes along with their dependencies to create containers (with *isolated* CPUs, *memory*, *block I/Os*, and *network*) that are collectively managed by underlying host OS kernel and can be seen as sandboxes running applications in isolated ways [1]. They do not need *init* as *PID1*. Container bundles necessary OS functionality to run a specific application along with the application itself to a unit visible to the host machine, in contrast VMs are opaque. To make it clearer, we explain Fig. 1 using Docker container that allows reproducible building process including Docker files (container files in Fig. 1), container image and runtime. It implements central repositories like Docker Hub (Image repository in Fig. 1), where developers upload their images, and other alternate registries for building (fetching process from external repositories, see Fig. 1) third-party application.

1) LINUX KERNEL SUPPORTS FOR CONTAINERS

Unlike hypervisor-based virtualizations, containers share same OS kernel that removes additional performance overheads. This has given containers a heavy industrial push with few serious security flaws (discussed later) and technical challenges [54], [55]. Following *two* are basis of light-weight process virtualization [64]:

① **Namespace**: Namespace has been introduced in Linux kernel 2.4.19 for isolating virtualized resources to a group of processes [64]. Each process can be associated with multiple *namespaces* of different types. These processes are provided with a customized view of system resources by kernel. Alteration to any *namespaced* system resource are confined only within related *namespace* that avoids influences of other *namespaces*. Six different types of *namespaces* like *mount* (MNT), UNIX Timesharing System (UTS), *PID*, *NET*, *IPC*, and *USER namespaces* are found implemented in modern kernels. MNT *namespace* isolates a set of *file-system* mount points for enabling *file-system* isolation such that different MNT-*namespaces* portray different *file-system* hierarchy views. UTS-*namespace* allows each container to be treated as an independent node by providing its own host and domain names. PID-*namespaces* efficiently virtualizes process identifiers such that each process is associated with *two* PIDs: *one* associated within process PID-*namespace* and *another* associated to host (globally unique). It helps to isolate one process from another in such a way that processes in one container can view only those processes that are having same PID-*namespace*. NET-*namespace* [65], [66] virtualizes network devices, ports, IP addresses, and IP routing tables. IPC-*namespace* isolates and confines IPC resources (signals, pipes, and shared memory). USER-*namespace*, introduced in Linux kernel recently, isolates user and group ID

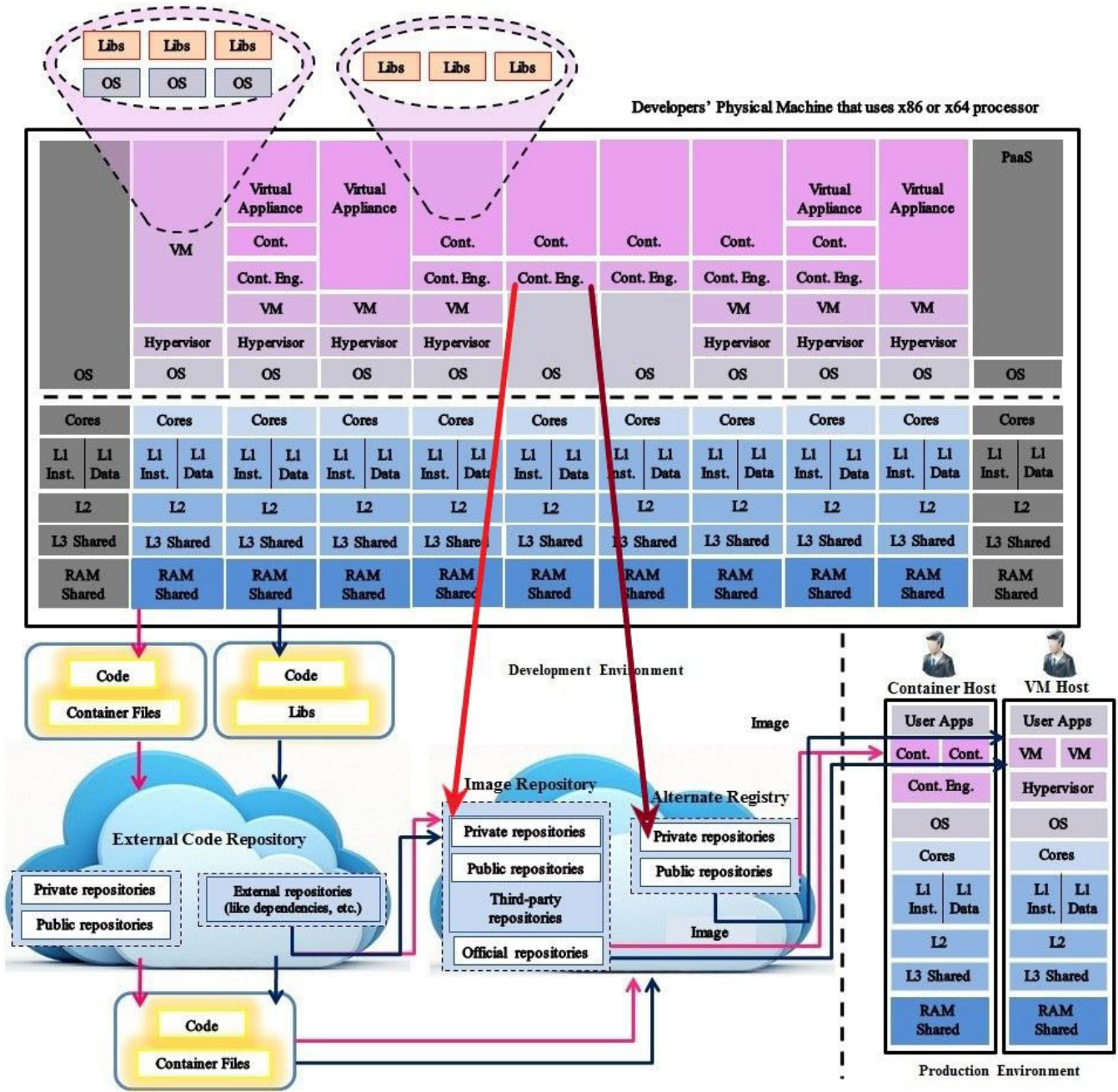


FIGURE 1. General diagram including different configurations of VMs and containers running on underlying hardware. It also present a rough idea about container production.

number spaces. It maps a `root` user inside a container with an unprivileged user on host thereby, providing full privileges for a process inside `USER-namespace`, but depriving it on host.

② **cgroups:** In Linux kernel, cgroups provides a mechanism for partitioning groups of processes (along with their children) in hierarchical order with controlled behaviors with the help of `cgroupfs` mount and `/proc/self/cgroup` file. cgroups does limiting (management, accounting and tracking) of memory, CPU, block I/O, and network resources

for the groups of processes and container instances for barring one instance from eating all host resources. Each `cgroup` subsystem provides a unified `sysfs` interface to simplify `cgroup` operations from user space. LXC, Linux-VServer, OpenVZ, Cells, Docker and LXC use cgroups for resource limiting [54]. Verma *et al.* [7] have reported that cgroups mechanism needs Linux CPU scheduler for achieving both low latency and high utilization for typical latency-sensitive, user-facing workloads at Google.

In short, *cgroup* limits amount of system resources for individual container instance, whereas *namespace* imposes instance-specific resource visibility.

2) APPLICATION CONTAINER VS. MACHINE CONTAINER

Containerization differs VMs as they do not need to include an entire process-supportive separate OS (several GBs in size), other than host OS (for *type-II* settings) to run hosted applications. However, containers like LXD, OpenVZ, etc., can contain complete *file-systems* with all normal OS and are named as *machine* containers. *Machine* container boots an OS and due to their small sizes, hundreds of containers can reside within a single core. Their performances are similar to bare metal with negligible latencies. On contrary, *application* containers (like *rkt*, *Docker*, etc., however complete OS can also run within a *Docker* container) run single process, app, or service with negligible latencies and near real-time performance. In short, *application* containers are better suited if requirement is to package and distribute application as components. On contrary, *machine* containers are useful while need is to install different libs, languages, apps, etc.

IV. ADVANTAGES OVER VMs

Despite containing many flexible software layers, including full OS image (a Linux or Windows kernel hosting a primary application running in user space, along with concurrently running secondary services), most deployed VMs ultimately perform a single function such as acting as a database or Web server that need only a part of OS for execution [67]. Unlike VMs, container hosting single application, only needs dependencies (packages, libs, bins and storages) necessary for application execution [50] in isolated memory, network and storage spaces [19]. Hence, containers are considered as more compact *single purpose appliances* than VMs. Many recent industrial surveys (conducted over 400 organizations) [68], [69] reported extreme *light-weightness* (Just Enough OS for running single application [70]) and *faster* spin up/down as *main* containerization advantages over VMs. Following are few specific benefits of modern containerization:

❶ Reducing *intra* and *inter*-datacenter storages and traffics by replacing VMs with evenly distributed containers [1], [71].

❷ Energy efficiency, resource utilization, application reliability escalation, and service cost reduction are achieved using containerization in datacenters [50]. Containerization allows cloud users [72] to see current versions of running application code and related dependencies, which enable them to look for differences between container editions and rolling-back to earlier version as and when needed.

❸ Container spin-up/down needs only few milliseconds, whereas VM needs few seconds for booting [1].

❹ Containers use *soft limits* for resource allocation (depending on resource availability) that enable hosted applications to use resources beyond pre-defined limits [50]. On contrary, dynamic increasing of resource limits are

relatively harder for VMs. *Soft limits* serve well during over-commitment and perform efficient resource utilization.

❺ Containerization increases developer efficiency, supports microservice architectures, enables apps to run on multiple cloud platforms, etc., [6].

❻ Containerization provides better storage utilization (with low storage *footprint* [73]) as we need less space for keeping multiple copies of specific container in datacenter network that leads to efficient proactive fault tolerance [71].

❼ During low-level system maintenance, we need to *migrate* only application container that runs desired application with necessary dependencies (very small size). On contrary, in case of VM, we need to migrate heavier VMs containing full OS with applications on its top.

❽ Providing much light-weight, faster process execution platforms [63] with a thin layer of necessary libs and bins for the application execution along with application code and supporting packages [1].

❾ Potential to develop, test and deploy applications to single or multiple of servers. Containers can also be inter-connected.

In short, hypervisor-based systems are well-suited for applications residing on same cloud requiring separate OS and hence, physical level abstractions are needed to distribute physical resources. On contrary, containers share host resources by running trimmed OS image with *root file-system* and necessary dependencies, which make them extremely light-weight and easily deployable to any hosts [15]. In spite of having several flexible layers, deployed VMs mostly perform single function that need only parts of OS for execution and hence, resulting in shear resource wastage, prolonged execution time and increased operation cost. This can be controlled using containers [67].

V. KEY ENABLING TECHNOLOGIES

A. KEY CONTAINER TECHNOLOGIES

A number of container technologies (see Table 3) are found in industrial applications are as follows: *Linux-VServer*, an oldest *Linux* container, implements *isolation* by employing mechanisms like kernel patch (changes between different versions of a source tree), installation of *userland* tools (various programs and libs that OS uses to interact with kernel: software performing input/output, manipulates *file-system* objects, application software, etc.), standard “*chroot()*” system call and POSIX capabilities (increases overall security) [74].

Unlike *Linux-VServer*, *OpenVZ* is placed above kernel namespaces to isolate all the resource subsets and consists of several user-level tools for virtualization, isolation, check-pointing and resource management. Similar to *OpenVZ*, *LXC* [75] provides container isolation by employing kernel namespaces and manages resources using *cgroups* and *blkio*. *LXC* can be further classified as *privileged* containers (container’s UID 0 is mapped to the host’s UID 0) providing secured access through MAC (like *SELinux*, *AppArmor*,

TABLE 3. Isolation strategies; backend OS and live-migration supports in modern containers.

	Process Isolation	File-system Isolation	Device Isolation	IPC Isolation	Network Isolation	Resource Limiting	Backend OS-support	Live Migration
Docker 2013	Process wrapping; <i>PID-namespace</i>	<i>MNT-namespace</i> ; <i>chroot()</i>	Device Whitelist Controller; device <i>namespace</i>	Container- specific IPC <i>namespace</i>	<i>Network namespace</i>	cgroups	Linux backend	Enabled
LXC/ LXD 2008	<i>chroot()</i> ; <i>pivot_root()</i> ; <i>PID-namespace</i>	<i>copy-on-write</i> ; <i>pivot_root()</i>	Device <i>namespace</i>	Container- specific IPC <i>namespace</i>	<i>Network namespace</i>	cgroups	Linux backend	Enabled
Cells 2008	<i>PID-namespace</i>	<i>chroot()</i>	Device <i>namespace</i>	Container-specific IPC- <i>namespace</i>	<i>Network namespace</i>	cgroups	Linux kernel (Android)	Enabled
Jails 2000	Storing pointer in process task structure	<i>chroot()</i>	Device <i>namespace</i>	Blocks SysV IPC object related system calls	<i>bind</i> filtering; restricting <i>bind()</i> system calls	RCTL	FreeBSD	Enabled but difficult
Zones 2004	Storing pointer in process task structure	<i>chroot()</i>	Device <i>namespace</i>	Using zone- ID	VNI for individual container; shared-IP zone	<i>resource pool</i>	Solaris	Enabled
Linux- VServer 2008	Storing pointer in process task structure	Secure <i>chroot</i> Barrier	Device <i>namespace</i>	Container- specific IPC <i>namespace</i>	<i>Network namespace</i>	cgroups	Linux backend	Disabled
OpenVZ 2005	Storing pointer in process task structure	<i>mount namespace</i> ; <i>chroot()</i>	Device <i>namespace</i>	Container- specific IPC <i>namespace</i>	VNI for individual container; <i>venet</i>	cgroups	Linux backend	Enabled from v2.15
Warden 2011	<i>PID-namespace</i>	<i>mount-namespace</i>	Device <i>namespace</i>	Container-specific IPC- <i>namespace</i>	<i>Network namespace</i>	cgroups	OS supports virtualization	Enabled
Garden	<i>PID-namespace</i>	<i>mount-namespace</i>	Device <i>namespace</i>	Container-specific IPC- <i>namespace</i>	<i>Network namespace</i>	cgroups	Backend OS + Garden API	Enabled
Zap 2002	<i>PID-namespace</i> ; IPC keys IPC IDs	<i>PID-namespace</i> ; per-POD private directory	Device <i>namespace</i> per-POD/dev	Container- specific IPC <i>namespace</i>	<i>Network namespace</i> per-pod virtual IP address	ProCess Domain Abstraction	Any OS that supports virtualization	Enabled

Seccomp, etc.) and *unprivileged* containers (UID 0 is mapped to an unprivileged user other than UID 0 on host) where above mentioned security measures are not mandatory.

Docker containers are extension to LXC, which implement faster deployment of Linux applications, make use of *namespaces* and cgroups to isolate and manage underlying operating environment and also include kernel-and application-level API, Docker to provide a better container isolation. rkt, another application container engine designed for state-of-the-art cloud-native production environments, is based on ProCess Domain Abstraction (POD), which is a collection of one or more applications or Application Container Images (ACIs) executed using shared resources. Resource isolation is provided at both POD-level and per-application level. Unlike Docker, which runs under PID namespace of Docker daemon, rkt gets executed under *init* system like *upstart* or *systemd* which facilitates running multiple processes inside a single container as opposed to Docker which runs a single process inside a container. rkt is more security focused and has support for Docker images and *kvm*. rkt fetches container images as non-root (important for security purpose); uses VM based isolation at stage 1 and supports *Svirt* in addition to default SELinux policy.

Apart from Linux, FreeBSD introduced containerization called Jail that can mitigate security related issues by partitioning a FreeBSD environment (partitioning *root* with processes, *file-system*, network resources) into a management environment, and on demand subset of Jail environments (having limited *root* access but privileged containers) rather than offering fine-grained access control techniques, simultaneously maintaining the existing UNIX security model.

This enables partition of FreeBSD machine into independent and isolated jails as well as creation of multiple users and a privileged user (*root*) in each jail. Solaris developed Zones to provide OS and hardware level abstractions to isolate applications from PM in terms of applications, device paths, network interface names, etc. Zap provides a virtualization layer on top of OS using PODs [76]. Process and *file-system* isolations are achieved using *PID-namespace*, IPC keys and IPC IDs. Cells is used as a virtual mobile smart-phone architecture enabling multiple virtual smart-phones to run simultaneously on same physical cellphone in an isolated, and secure manner [77]. Cells [77], [78] implements isolation using kernel *namespaces*, cgroup and *chroot()*.

B. CONTAINER MANAGEMENT & ORCHESTRATION

Container management strategy not only simplifies administrative workload of creating, deleting, deploying, configuring and scaling of large number of containers on host, but also automate deployment of container-based application to various heterogeneous infrastructures with increased interoperability [1], [79]. Noteworthy features of container management tools are:

- ① Managing multiple containers as one single entity, efficient container organization, sensible launch and replication of container instances over geographical locations for fault-tolerance.
- ② Efficient cluster formation for maintaining high availability, better resource utilization, load-balancing, automatic data backup and disaster recovery.
- ③ Scheduling container execution for better resource utilization.

④ Good orchestration tool should do Container auto-scaling that automatically change the number of running containers, based on CPU utilization or other application-provided metrics. Kubernetes uses Autoscaler for deployment and Kubernetes 1.2 adds alpha support for scaling containers.

⑤ Volume management should be done for controlling persistent storage. Kubernetes provides different types of volumes, each of which is implemented as a plugin. Current plugins include Google Cloud Platform volume, AWS Elastic Block Storage volume, Ceph block device, Empty dir (backed on the node where the PoD runs, thus it just leverages local storage from that node; it exists on that node until the PoD is removed).

⑥ Resource usage monitoring (CPU and RAM) is required at different levels like container level and logical group level (for example, Kubernetes PoDs) and at the cluster level.

⑦ Automatic reporting of issues found in container orchestration and production. It requires a strict monitoring mechanism that includes few core capabilities like ability to gather detailed data and analyze it to spot possible future failures; ability for taking automatic action.

⑧ Service discovery module should be a part of good orchestration tool. Kubernetes supports *two* primary modes of finding a service through environment variables and DNS.

⑨ Rolling system updates should be allowed by a good orchestration tool. Kubernetes allows rolling system update for containers.

Commonly used open source container management tools are OpenShift, Docker Swarm Manager, Kubernetes and Mesos Marathon. Swarm is an open source infrastructure independent Docker management platform and supports applications by aiding complete stack portability throughout their life-cycle. It enables predictability of a product during all stages of development, testing and deployment. It can easily integrate with existing systems such as storage, network and authentication. Kubernetes is an open source platform that manages microservices or containerized applications across distributed cluster of nodes. It is responsible for auto-placement, auto-restart, auto-replication, CPU usage based scaling. Both Kubernetes and Docker follow master-slave architecture. Mesos cluster architecture consists of one or more Mesos-master (one active *master* and other *standby*) and several Mesos-slaves. It uses Apache Zookeeper for electing an active master from standby masters and Marathon for service discovery. OpenShift, a PaaS solution provided by Red Hat for enterprise application utilizes Kubernetes and Docker Swarm as underlying container management engines for automating deployments, health management, scaling, application development and entire container life-cycle maintenance.

An important conceptual difference between orchestration and service discovery problem (discussed in Section II-A) is

that former implies the process of deploying, monitoring and dynamically controlling the configuration of multi-container packaged application in the container cloud, whereas service discovery problem occurs because services cannot be counted on at a given moment and the consumer service needs to locate the provider service for successful execution [41].

C. CONTAINER NETWORKING

Microservices running on containers heavily rely on underlying networking resources for service provisioning as well as communications between different microservices (or *inter-container* communication, since each microservice runs on individual container) [80]. However, providing reliable networking infrastructure from container perspective is done as follows:

① *Fully isolated networking view for applications running within containers* [80]: It is provided by UNIX Time Sharing (UTS) and network *namespaces* by enabling isolated network interfaces, routing table, IP tables, loopback device, forwarding rules along with kernel modules like *veth*, *ipvlan* and *MACVLAN*, thereby allowing containers to act as an independent system [80], [81].

② *Discovering and addressing for replicated containers throughout their life-cycle specifically in clustered environment* [80]: Container clusters are typically a bunch of tightly coupled containers deployed on same PM, sharing same resources and generally placed inside same network *namespace* for enabling inter-cluster communication via local host. Container clusters provide a persistent network even if individual containers in cluster are scaled up/down, restarted, created/destroyed. Persistence network can be achieved by assigning an IP *per cluster* or using *dynamic port allocation and mapping*.

Container network design is also influenced by number of host machines required to deploy applications and can be classified as *single host container networking* (categorized in several modes like bridge, host, container and no networking) and *multi-host container networking* [82] that considers connections between container/host and inter host as well as its performance and security. It makes use of overlay networks (creating a distributed network across hosts on top of host-specific network) as well as network plug-ins for third-party providers. Number of popular multi-host container networking options are Flannel, Calico, Canal, OpenVPN, and Open vSwitch (OVS) [82]. Container network provisioning and management can also be optimized by adopting Software Defined Networking (SDN) paradigm [83], [84]. SDN capability can be fully leveraged by using OVS [83], where several containers on a host can be connected to OVS bridges that can be connected to other OVS bridges residing on different hosts. SDN controllers like OpenDaylight (ODL) or Open Network Operating System (ONOS) can be employed to visualize, and manage OVS switches [83].

VI. CONTAINER SECURITY ISOLATION

Container security problems start with sharing common host kernel among different instances without installing proper security and isolation mechanisms, whereas, VMs run on separate kernels and hence, considered as more secured option [1]. Containers running on the top same host OS significantly expand the attack surface in comparison with VMs. For example: privilege containers have the root access to the host OS and hence the attackers can evade the privacy of host OS and collocated containers. Hence, in many cases, cloud providers offer containers running inside VMs because of VMs' stronger security assurance toward protecting data center assets against external customer code. Apart from that, in multi-user environment, where multiple user can deploy containers and able to run malicious codes hidden inside container images downloaded from official or public repositories. Sometime, container images downloaded from official repositories are not updated for several days and may contain vulnerabilities. Moreover, vulnerabilities mainly propagate from parent images to children images. Direct adversary is able to sniff, inject malicious code snippets, block or modify access privilege and network system configuration of the in-production containers (by gaining root privilege on a container and them making DoS to other collocated containers), in-production host OS by gaining access to the critical host files from a compromised container), and in-production Docker daemons (by lowering default security parameters necessary for Docker containers from a compromised OS) and the production network (redirecting network traffic from a compromised host OS). However, an indirect adversary has the same capabilities but uses Docker ecosystem to reach the production environment. Hence, Docker's default configuration on local systems following recommended usages (microservices approach, i.e., container must host only one service) has been found relatively more secured due to its limited communication between container and host, which provides better isolation [16]. With wide-spread usage (shipping complete virtual environment with a container), Docker becomes more attack prone. Moreover, Docker image from an untrusted repository increases vulnerability in manifests.

The success of application containers can be partly attributed to their ability to share resources with the underlying host. However, in the absence of strict isolation mechanism, the characteristics that have contributed to the success of application containers can also be seen as factors limiting their possibility of widespread industrial adoption, since enterprise IT administrators cannot implement the various- and often finegrained-security policies they are required to abide by. Hence, containerization needs robust isolation mechanism installed within to defend against various adversarial activities and to maintain its exponential growth in various industrial applications [5], [54], [85]. Let C_i ; $i \in \mathbb{N}$ be individual container; $\mathcal{C}$ and $\bar{\mathcal{C}}$ be container classes controlled by legitimate users and adversaries respectively such that if $C_i \in \mathcal{C}$ then $C_i \notin \bar{\mathcal{C}}$ and vice versa.

A. PROCESS ISOLATION

Process isolation aims to isolate various processes running within different containers on same (or different) host such that $C_i \in \bar{\mathcal{C}}$ ($\subset \mathcal{C}$) does not influence $C_k \in \mathcal{C}$ ($\subset \mathcal{C}$) using signals and interrupts. Moreover, process isolation can successfully restrict information duplication whilst process and container need similar information. However, often memory of a process running in $C_k \in \mathcal{C}$ can directly be accessed by a debugger process using `ptrace()`-type system calls. Containers implement process isolation using PID namespace, `chroot`, etc. [65].

LXC/LXD implements *process isolation* using `chroot` that can change the working `root` directory of a process and its sub-processes. Entire `root file-system` can be copied to a new location using `pivot_root`. `Pivot_root` facilitates users in specifying old and new `root` directories; copying files from old `root` to new `root` directory; and lastly, making new `root` directory as current `root`. LXC and cells also use `PID-namespaces`, mainly a part of mainline Linux kernel, to implement process isolation. `PID-namespace` is a strategy to group processes for controlling their ability to see (like via `proc pseudo-file-system`) and interact (like sending signals) among themselves. `PID-namespaces` also provide PID virtualization that enables *two* processes in different `PID-namespaces` to having same PID. `PID-namespace` allows each container to have its own `init`-like process (with PID 1) that can cause all processes within a *namespace* to be discontinued if it is terminated. Process with PID 1 assists admins in completely shutting down a container upon detecting any adversarial activity.

Docker implements *process isolation* using process wrapping within related *namespace* and limiting their visibility to the processes running in other containers or hosts [54]. This is achieved using `PID-namespace` that separates the host PID space from that of container. Since, structures of `PID-namespaces` are hierarchical, a process can only see other processes within own namespace or "children" namespaces. If a new *namespace* is created and assigned to a container, host can observe and affect processes inside new `PID-namespace` of the container, but processes inside container cannot observe or do anything to other processes running in host or in other containers. Hence, it will be harder for attackers as they have very little understanding/reach about other processes running in other containers located in same or different hosts.

Maintaining a separate structure that describes a container and storing a pointer to that container in process task structure are found to be implemented in FreeBSD Jails, Solaris Zones, OpenVZ and Linux-VServer for *process isolation* [85]. On contrary, upstream Linux kernel has observed a bit different approach of grouping various kernel resources (kernel structure referring to shared physical/virtual devices, system resources like CPU, memory, etc.) into independent namespaces and utilizing these namespaces to build containers. This approach increases complexity, but provides flexibility to choose a combination of namespaces

that fits appropriately for desired use case. It also facilitates gradual introduction of *namespaces* to an existing system, similar to upstream Linux kernel, which also aids in testing and verification of implementation [65].

In addition to ability of virtualizing and isolating PIDs, upstream Linux kernel also capable to virtualize and isolate user and group identifiers by means of user *namespaces*. Typically, root user possesses all privileges to handle different system administration tasks and has ability to override all access control restrictions. However, containers should run as non-root. Therefore, implementing Linux user *namespace* implies a given Linux capability like authorizing an action, within that *namespace*: for example, CAP_SYS_BOOT capability within a container allows authority to reboot that specific container and not host. On contrary, capabilities like CAP_SYS_MODULE cannot be granted safely for a container in any meaningful manner. When a particular process tries to execute an action guarded by such capability, kernel always verifies whether that process possesses this capability in host user *namespace*. All existing Linux containerization solutions support an option of initiating new user *namespace* for each container, but requires manual configuration like user identifiers mapping between host and container.

B. FILE-SYSTEM ISOLATION

It protects benign containers ($C_k \in \mathcal{C}$) from being compromised by malicious containers ($C_i \in \bar{\mathcal{C}}$) by keeping C_k indifferent from C_i *file-systems* [85]. Another aspect of *file-system isolation* is isolating *file-system* among various co-located containers that, in turn, minimizes amount of shared data. Amount of sharing often remain scenario-specific like duplicating complete OS setup within containers may not often be cost effective and against container-usage perspective [64]. Hence, container needs to securely share some parts of host *file-system* for which *file-system isolation* becomes a must.

Linux containerizations use *mount namespace* (inspired by *private namespace* [86]) for *file-system isolation* by separating *mounts* between host and container [54]. Following this Linux systems create containers within new *mount namespaces* and internal mount events become active within specific containers. It restricts all mount events within a container to influence other events in different co-located containers, thus provide isolation. Precisely speaking, *mount namespaces* should not be viewed as an extra security add-on as containers already inherit *file-system* mount from their parents and hence, can directly view parent *file-systems*. This can cause a serious security threat in presence of co-located compromised container. Docker mitigates this issue by removing write permission to mentioned *file-systems* from containers, and restricting all containers processes to re-mounting any *file-system* within containers by removing CAP_SYS_ADMIN privilege from containers [87]. Above all, Docker employs copy-on-write *file-system* that restricts other containers, created on the same image, to modify any specific container *file-system*. Among other popular ways

for achieving *file-system isolation*, chroot() alone has been found to be implemented in Cells, whereas, Linux-VServer uses Secure chroot barrier for achieving the same [88]. FreeBSD Jails and Solaris Zone use chroot() like system calls that eliminate above security threats by allowing different allow.mount options or by restricting container internal processes from mounting/un-mounting *file-systems* within container [89], [90].

LXC mounts *copy-on-write* current host *file-system*. System's /home directory is not imported in LXC container by default and, hence, any modification to container *file-system* will only stay within container. Consequently, host *file-system* remains unaffected. This feature helps LXC container to potentially *isolate* malicious processes from host like installing (apt-get) software packages (will not be installed in container because of imposed *file-system isolation*), chroot system for running servers, etc. LXC also uses pivot_root() that is mainly, used while booting to changing the mount-point from a temporary root *file-system* to older root. It can also move mount-point from an old root to a directory under new root *file-system*. Older root *file-system* can be unmounted, while mounting within a *namespace* that provides container-specific *file-system isolation*.

C. DEVICE ISOLATION

For UNIX family of OS, devices are treated as special security-sensitive files that provide interfaces to host device drivers as inappropriately shared devices may lead to severe security threats. Typical UNIX configuration separates devices from rest of the *file-systems* and places their inodes in /dev, whereas in Linux, udevd daemon process issues mknod system call upon receiving event from kernel [86]. A strong motivation behind the possible enforcement of *device isolation* in containerization can be if container has default privilege of using important devices like /dev/mem, /dev/sd*, /dev/tty, /dev/kmem, etc., will lead to serious and sensitive information disclosure. Hence, *device isolation* must be implemented to maintain container isolation by allowing only “safe set” of devices, classified by Reshetova et al. [85] as ❶ *Purely virtual devices* [85] like pseudo-terminals, virtual network interfaces, etc., must be explicitly created for individual containers and must not be shared among containers; ❷ *Stateless devices* [85] are securely shared among containers and hosts due to their stateless nature; ❸ *User namespace-aware devices* [85] can be safely exposed to the containers as they impose namespace specific user restrictions. This is currently not implemented but can be a potential security parameter for foreseeable future. Cells implements *device isolation* using *device namespace*. Docker implements *device isolation* using *Device Whitelist Controller* feature that limits set of devices connected to a container [54]. Moreover, Docker mounts container images with nodev system call and limited privileges (stops using any device) that restricts an already created device node to communicate with kernel.

D. IPC ISOLATION

IPC can be viewed as a set of rules (conventional SysV IPC primitives like semaphores, shared memory, etc.) defined for sharing inter-process data [85]. IPC isolation is necessary for controlling communications among processes running in containers only through a set of pre-defined IPC resources and restricting processes from modifying data of other co-located containers and host. It prevents $C_i \in \tilde{C}$ to access $C_k (\in C)$ -specific data transmitted over IPC channels.

IPC isolation is implemented in Linux containers (Linux-VServer, LXC, OpenVZ, Docker, and Cells) using container-specific IPC *namespace*, where process within an IPC *namespace* is prohibited from reading/writing IPC resources in different IPC-*namespace*. Cells needs *namespace* support from Binder system (Android's primary IPC method) for which, IPC *namespace*-specific context managers are created that resolve Binder address within a specific *namespace* providing complete Binder address isolation among various containers. Solaris Zones adds Zone ID to each object based on Zone ID of its parent process, and, hence, zone-specific objects remain inaccessible to processes from other zones. Administrator (in global zone) can access objects from other zones. FreeBSD Jails blocks those SysV IPC object-related system calls that are issued from a jail. SysV IPC mechanisms are allowed for jailed processes with the help of `allow.sysvipc` option, but it does not provide any isolation between jails.

E. NETWORK ISOLATION

It is required to prevent containerization from various network-based attacks. Virtualized view of network stack is often needed for server consolidation and resource management. FreeBSD Jail and Linux-VServer implement network isolation on Layer 3 using *bind filtering*, which restricts a `bind()` call from within a container to a set of specified IP addresses as a result of which processes send/receive packets to/from these addresses with minimal performance overhead. Solaris Zones and OpenVZ implements network isolation providing Layer 3 virtualized network interface (VNI) for every individual container. This strategy is a bit more flexible than *bind filtering* as VNI allows configuration of various traffic control settings. OpenVZ's Layer 3 implementation is named as *venet*, whereas that for Solaris Zones is termed as *shared-IP zone*. Docker implements *network isolation* using *network namespace* that creates virtual ethernet bridge, called `docker0`, in the host machine, which automatically forwards packets between network interfaces. Docker establishes a new virtual ethernet interfaces with unique name and connects with the bridge while creating new container. This interface is also connected with container's `eth0` interface that enables container to send packets to bridge.

F. RESOURCE LIMITING

Containerization often needs imposing sensible limits on available and allocated physical resources to individual containers for preventing DoS attacks. Several resource limiting strategies are found in containerizations. Docker employs `cgroups` for controlling various physical resources like CPU, RAM, I/O, devices, etc.; ensuring individual container with desired amount of resources that, in turn, prevents any specific container from devouring all available system resources. In this way, Docker prevents DoS attacks. LXC, Linux-VServer, OpenVZ and Cells also use `cgroups` mechanism for resource limiting. FreeBSD Jails implements individual *jail-resource limiting* by employing Hierarchical Resource Limits that defines actions like stopping new resource allocation, signal or notification (SIGHUP, SIGKILL, etc.) sending, etc., while a pre-defined resource limit has been exceeded. Solaris Zones utilizes a resource partitioning technique, termed as *resource pools* (where a dynamic pool allows resource allocation based on system loads), that allows defining a set of resources like a physical processor set for individual zone. Cells and Linux-VServer implement container resource limiting using `rlimits` (a conventional resource management technique found in FreeBSD), though, `rlimits` does not allow users to specify resource limit for individual container. However, custom resource management extensions are found to be implemented in OpenVZ and Linux-VServer.

G. ADVANCED Linux SECURITY MODULES

This subsection talks about different advanced Linux security modules that enhance container security by imposing MAC over traditional Linux and POSIX Access Control Lists' (ACL) DAC [51] implementation (owner or group specific restriction on object access). DAC provides full discretion over all system objects to respective owner by checking/ comparing process UID and GID with that of requested file for `read/write/execute`. Moreover, in DAC, controls over objects are treated as discretionary that enables a subject with a certain access right to passing permission/right to any other subjects. Hence, complete control over all in-account objects switch to adversaries as and when account gets compromised. Modern Linux kernel implements MAC (security mechanism for restricting individual resource owners ability of granting or denying access to resource objects in a *file-system*) or policy based access control layer over DAC to add constraints on DAC using following modules:

❶ **SELinux**: A security architecture that implements MAC over default Linux DAC for controlling object access. DAC provides complete discretion over all system objects to respective owner and hence, control shifts to adversaries as and when the account gets compromised. In contrast, MAC enforces policy-based access and transition rights control to every individual user, application, process and file present

in system. SELinux governs these system objects using very complex and flexible security policies (*strict* or *lenient* as needed), independent of specific security labels and contents, for providing secure process, *file-system*, and user. SELinux provides clean separation of very complex policy from enforcement with well defined policy interfaces; individual labels and controls for kernel objects and services; caching of access decisions for efficiency; supports for policy changes. controls over various system objects, sockets, messages and network interfaces. It does not require labelling or re-labelling of *file-systems* and provides control over process initialization, inheritance and program execution. This mechanism is relatively harder to deploy and use.

② **AppArmor**: A *path-based* Linux kernel security module (enforcing MAC) for binding access control attributes to programs rather users by using *two* profile (*read* and *audit*) modes called *enforcement* (enforcing defined policies to profile) and *complain* (reports policy violation attempts). It allows mixing of modes and often has a lower barrier of entry than other popular MAC systems. AppArmor provides easy deployment; supports for YaST-based GUI tools to help development, deployment and maintaining application security policies; protects third party applications from internal and external security threats by enforcing appropriate application behavior. AppArmor supports developers in scheduling detailed event reports and configure alerts based on user defined events. Unlike SELinux, AppArmor attaches labels to all files, processes and objects.

③ **Seccomp**: This Linux kernel security module allows process to make an one-way transition into a *secure* state, where it cannot make any system calls except `exit()`, `sigreturn()`, `read()` and `write()` to already-open file descriptors (like a system call filter, called Seccomp filter). Seccomp filter allows processes to specify filter for incoming system calls, which reduces total exposed kernel space to specific applications. It blocks some dangerous activities like forced umounts, `kexec`, kernel module loading and unloading, and open-by-handle-at system call.

④ **grsecurity**: It is an intelligent and robust RBAC system to generate least privilege policies for utilizing multi-layered detection, prevention and containment model. It also deploys `chroot` hardening; `/tmp` race prevention, extensive auditing; prevents arbitrary code execution in kernel; does stack, `lib` and heap randomization; protects processes from exploitable null pointer vulnerabilities; restricts users from viewing others processes. It is very easy to deploy and use

H. FEW BEST PRACTICES FOR MAINTAINING CONTAINER SECURITY

Above security mechanisms are found installed in Linux containerizations. However, user/developer wisdom would often play final role for maintaining container security. Following are few best practices towards comprehensive adoption of container security:

① **Use Trusted Images**: Developers often like assembling container images rather than building from scratch. Hence, it is always advisable to use images from trusted repositories.

② **Container Secret Managing**: Container secret should not be exposed to many users and processes and hence, it is advised not to store them on disk or not to expose them to host level. Secrets should only be made accessible to relevant containers during runtime.

③ **Secure the Runtime Environment**: Always, use *namespace* and *cgroups* for isolating container *access* and *control*.

④ **Vulnerability Scanning**: Preventing images with known vulnerabilities from running in production environments using trusted and efficient vulnerability scanning tools.

⑤ **Enabling MAC**: It is always advised to enable MAC (by adding different advanced security modules discussed in Section VI-G) over traditional DAC for comprehensive adoption of container security.

⑥ **Running Containers as Non-Root**: Containers must run as `non-root` for their security escalation.

VII. KEY CONTAINER APPLICATIONS IN INDUSTRIES

Noteworthy industrial container applications are as follows:

① **Containerization in Gaming**: “Pokemon GO” was deployed on Google Cloud Platform using Kubernetes clusters. Since its inception, Niantic targeted 1x traffic with expected 5x traffic at worst case, but actual traffic was nearly 50x. Despite this sudden traffic surge, “Pokemon GO” was able to efficiently serve millions of users due to *scalability* and *flexibility* of Kubernetes. Kubernetes monitored traffic and adapted itself by creating/destroying containers according to traffic flow fluctuations. Container-based development, offered better control, faster and huge connections with an exceptional Quality of Experience to users. “iPokemon” is deployed using Amazon EC2 cloud and LXD containers for distributed and light-weight computation [91].

② **Containerization in Healthcare**: Possible containerization application could be *gateways* in Cloud-of-Things (IoTs, Edge Computing, Web of Things) due to its ability of working as an end-point for user data presentation; light-weight and dense service deployment with negligible performance impact [92], [93]. Dean *et al.* [94] developed an inexpensive cloud application for processing large protected healthcare data (using Docker for gateways), which enables necessary storage and processing related features. Moreover, healthcare applications should provide facilities like location-awareness and on-the-go provisioning with minimum latency, may be difficult for a single cloud provider. Thus, Palesandro *et al.* [95] proposed a multi-cloud healthcare system (using LXC) that provides flexible provisioning; optimized resource allocation; inter-operability support; resource abstraction and security. In other works, McPadden *et al.* [96] and Boonma *et al.* [97] used containerization for deploying data analytics platforms for analyzing healthcare and biomedical data.

③ **Containerization in Network-Function Virtualization (NFV):** NFV [98] is a trending concept where various network functions like *routing*, *switching*, *firewalls*, etc., are virtualized in a software applications to significantly minimize CAPEX and OPEX [99]. Traditional NFV approaches adopted full-fledged VMs which made them extremely heavy and were unable to perform complex tasks [100]. In one variant, Li *et al.* [100] presented light-weight deployment of deep learning models within NFV containers. Pattaranantakul *et al.* [99] developed a security orchestrator that enables dynamic generation of access control models and policies for different tenants across different NFV layers and multiple data centers. Tseng *et al.* [101] amalgamated hypervisor with containerization to construct an integrated virtualization *fog* platform (for improving service scalability) for industrial application deployment using NFV. In a similar application, Boubendir *et al.* [102] have developed an on demand dynamic model that implements network as part of NaaS using SDN-enabled NFV. Riggio *et al.* [103] proposed a multi-access network OS that integrated SDN and NFV in a single light-weight framework to manage and orchestrate network services over distributed NFV architecture.

④ **Containerization in Microservices-Architectures:** Microservice consists of multiple independently deployable, scalable and testable (platform and technological stack independent) small services that together constitutes single complex software system [25], [71], [104], [105]. Containers being light-weight and portable are the first choice for implementing microservices architectures [71], [105]. Containers are capable enough to produce same environment in both production and development thereby curbing conflicts between operations and development teams [104]. Migrating monolithic architectures to microservices provides benefits like adaptability, technology lock-in, reduction in time-to-market and better development of services [104]. For example, Balalaie *et al.* [104] migrated a monolithic application “Backtory” to microservices architecture for providing chat-as-a-service. Service instances were deployed using Docker containers due to its capability to provide lower overheads and produce same environments without changing source codes or container images. Another example can be Netflix that pioneers migration from monolithic to microservices [106] for improving robustness, availability, and scaling. According to Mohamed *et al.* [21] and Tosatto *et al.* [49], the adoption of container technology was influenced by SOA in cloud to manage service life-cycles of deployed applications. Mohamed *et al.* [21], introduced a micro-container (container with single deployed service) offering web services.

⑤ **Containerization in Scientific-Workflows:** Scientific workflow requires specific configuration, OS-specific libs and packages that can vary from system to system and may not be replicated in due to external dependencies incompatibility [10], [49]. Gerlach *et al.* [10] developed technique using containers for deployment and execution of scientific workflow applications in an isolated, portable and

self-contained environment that can be distributed efficiently across a wide range of computing platforms. Zheng and Thain [6] used containerization for providing a well-defined run-time for scientific workflows running in distributed architecture. They have also pointed out possible trade-offs among isolation, performance and consistency.

⑥ **Containerization in Edge Computing (EC):** High latency, network congestion and network bottleneck are considered to be some of the potential problems with cloud computing, for which there is a desperate need of switching from centralized to decentralized paradigm, where edge computing could be a potential solution [73]. EC is the practice of processing data near the edge of network, where the data were generated, instead of in a centralized data processing center. In other words, edge computing promotes distributed open IT architecture, featuring decentralized processing power, enabling mobile computing and IoT. Ismail *et al.* [73] evaluated Docker as EC platform with four fundamental criteria, called deployment & termination; resource & service management; fault tolerance; and caching.

Proliferation of connected objects in cloud has often been suspected as a reason behind data confidentiality and network performance related issues, for which, *fog* can be thought of as potential solution. In general, intelligent systems deployed in *fog* receive requests based on system context and user-objectives and the analytic task is divided into segments that are offloaded to surrogates (*fog* nodes to offload portions of large analytic jobs) and final results are returned to specific user application [107]. Context manager node monitors surrogates’ status and provides context information to other components for decision making. Dupont *et al.* [108] have developed one such platform (called Cloud4IoT) for performing *horizontal* (roaming) and *vertical* (offloading) migration of IoT functions using a *three-tier* (cloud, edge and IoT gateways) Kubernetes cluster. In another work, Carella *et al.* [109] presented an architecture based on the Open Baton Management and Orchestration (MANO) framework combining different infrastructural technologies supporting deployment of container-based network services even at the edge of network [109].

⑦ **Containers in Other Applications:** Wu *et al.* [20] adopted containerization to track full motion videos by partitioning the video stream into frames and dynamically allocating them (optimizing computing resources) to various containers depending on frame production and available resources for processing. Jimenez *et al.* [22] showcased the use of containers for reproducibility (in terms of workload, system and result) of research outcomes, by creating experiment-specific execution profiles that focuses on local and distributed storage. According to Saez *et al.* [23] container-based virtualization can be a perfect solution for rapid and efficient building of middle-ware instances that are extremely lightweight and optimally configured. Containers are excellent tools for cloudifying the existing middlewares and making them as part of larger PaaS offerings and hence, the authors devised a dynamic tailoring and deployment process for service

middleware solution in the form of Enterprise Service Bus (ESB) geared towards cloud environments.

VIII. RESEARCH CHALLENGES AND FUTURE TRENDS

Few challenges resisting containerization from gaining desired rapid momentum are as follows:

A. COMPLEX NETWORKING

Containers are often created and destroyed on demand for better scalability. This leads to the following problems:

❶ Problems may occur due to the inability of using IP-address as unique identifier during large scale container deployment that could have a good possibility of assignment of same IP-address to multiple container instances. Log maintenance could be much harder with non-unique IP-addresses.

❷ IP-address assignment during large scale container deployment remains a big challenge due to constant risk of IP collision during *simultaneous requests*. This could often make log maintenance tougher. Possible solution could be creating containers within VM and allocating unique identifier, combining VM IP-address with container port number, to each container [12].

❸ Containers often use NAT for network connectivity that can make troubleshooting difficult for large scale implementations. NAT usage can be avoided by directly attaching containers to host network interfaces by sharing LAN and obtaining IP-addresses from its DHCP server or providing with static LAN address. NAT can also be avoided by turning host machine into a virtual router which can route a prefix to the containers thereby using IP-address as a unique identifier for containers. Although, this configuration offers less complexity, it may compromise security as all the Layer 4 ports gets exposed. Moreover, as the number of containers in rack increases, the number of MAC addresses in Layer 2 forwarding table of Top of the Rack (TOR) also increases, which may hamper traffic switching.

B. COMPLEX ORCHESTRATION & MANAGEMENT

Despite several innovative advancement, container orchestration still needs further research in the following challenges:

1) MONITORING, PROFILING AND CHARACTERIZATION [40]

Container monitoring includes monitoring of container ecosystem and container engine/platform. Monitoring tools and techniques for OS and application levels do not allow wide range of QoS and health state metrics for containers. Above all, there is no unanimously agreed definition of QoS metrics for container environment. Docker offers “`docker stat`” command to check CPU and memory utilization for each running container.

2) PERFORMANCE MODELS [40]

There is a desperate need of efficient performance validation and energy consumption models for distributed containerized applications, which should be included in the modern

orchestrators for better container management. Performance models are widely used in autonomic computing as representation of the system that must be adapted and as a tool to determine the reconfiguration actions needed to maintain the desired level of service. An alternative approach is the use of machine learning techniques to determine the more appropriate reconfiguration action.

3) ADAPTATION MODELS FOR CONTAINER ORCHESTRATION [40]

Industry needs a robust container orchestration framework for QoS-aware, energy-aware and legislation-aware optimal adaptation for container orchestration. This framework define system models, QoS, energy and legal constraints, to find optimal adaptation policies for container orchestration at run time.

Apart from them, few other challenges regarding container deployment and orchestration are as follows:

4) COMPLEX LIFE-CYCLE MANAGEMENT

This is a real problem during stateful container deployment and orchestration. This is because, more often than not, stateful distributed services can have complex deployment steps with multiple dependencies (for example: kubernetes deployment has 21 steps). Moreover, the orchestration operators have to manage orchestrator in real time and also have to take care about various upgrades, failures, configuration changes, scaling, etc.

5) RESOURCE ALLOCATION AND UTILIZATION

Deploying multiple distributed systems often results in sub-optimal resource utilization.

6) MULTITENANCY

Multitenancy becomes much harder with inclusion of multiple tenants with their own requirements and isolation & security because of the specific questions as follows:

❶ It is very to monitor monitor/resist any specific tenant from hijacking huge resources of a cluster, which might influence the performance of other co-located tenants.

❷ It is also hard to be sure about a specific tenant not breaching the privacy of other colocated instances.

C. PERSISTENT STORAGE

Persistent storage can be viewed as storage volume that remains available beyond specific container life.

❶ Persistent Storage *problem* for container points to the inability of stateless containers to act like stateful applications regarding persistent storage (like database) due to their extremely short life-span. Again, providing a huge and complex stack of storage may introduce *contentions*, which, in turn, may hamper the lightweight feature of containerization and, hence, issue regarding persistent storage remains [73]. Container storage requires efficient data mobility, enhanced data security and optimal performance. Shared storage was once considered to be a possible solution, but

with the increase of public, federated cloud and considering ephemeral lifetime of containers, the strategy got lost in time. Automated mapping of Logical Unit Number (number for identifying a logical unit or a device addressed by (SAN) protocols [110]) and volumes on physical arrays to physical hosts and to finally individual container evolves as one possible solution (example: IBM Docker Volume plug-ins enable persistent storage and retrieval of a resource within a container). Another possible solution is to use containers themselves as the storage platforms (also called *storage containers* [111]) as the communication path between the application and the storage will become very light-weight (it will be a simple communication between to processes) due to the elimination of I/O stack. But there are still issues with these solutions, e.g., a data volume cannot be attached to new spin up container or existing running container and hence, the resources that volume consumes are not utilized, which may not be an optimum solution. Moreover, this voids the basic containerization principle of being lightweight.

② **Persistent Storage Management Problem:** Containers are often created and destroyed and it would often be difficult to keep track of the data under use. Hence, for containerization without hampering data access ability the challenges faced by current industries are to build parallel systems for various stateful applications like legacy codes, databases, and information systems. Legacy architectures are often found complex with extremely low API support, hence, desperately needed persistent storage facility for successful containerization.

③ **Data Transfer Problem:** Lack of persistent storage utilities within containerization may increase difficulties during secure data transfer between locations or even in clouds. This problem increases because of unavailability of powerful tools and platforms that provide infinite scalability, significantly high performance, security and complete application control.

④ **Time & Money:** Industries cannot replace their important and invaluable legacy applications overnight with containerizations as it requires huge *time* and *money*. Consequences after replacing with containers can even be costlier than continuing with legacy applications.

D. CROSS DATA CENTER OR MULTI-CLOUD SUPPORT

It is essential for edge clouds provisioning and can be classified based on their architecture [112]: *multi-data* center clouds consisting multiple tightly coupled data centers run by a single cloud provider entity; *multi-service* clouds combining loosely coupled services from different cloud providers; and *distributed* edge clouds providing edge resources in a highly dispersed manner. Running containers across multiple clouds encounters *challenges* like location-awareness; replication; container scheduling and placement; dynamic and flexible adaptation; resource discovery and coordination; and service container mobility across multiple vendors. Edge cloud provides a localized view and hence, attributes like location awareness, low latency and mobility support are

stringent for managing cloud end points with ample virtualized resources [113].

Edge cloud [114] can be deployed a virtualized infrastructure called micro-clouds [115] that can provide different services but on a small scale and a seamless data transfer between these micro-clouds to provide an appropriate end-user access [113]. Combination of micro-clouds and a centralized cloud architecture can suffice all issues mentioned above as well as provides quick access to the services and faster fault tolerance. Such multi-clouds (cloud of clouds) architectures [116], [117], reduces possibilities of outage, security threats, loss of control and data systems, vendor lock-in and provides high availability, high reliability, flexibility and easy scalability [116].

① **Hybrid Deployment Problem:** Multi-cloud deployments faces issues like conflicting hybrid cloud architectures, vendor-specific devices and complex management. Cloud providers try to solve these problems for their specific clouds whereas, service providers fix the problems pertaining to their services only. Thus, multi-cloud environments result in incompatible APIs, different dashboards and siloed monitoring-all adding up to inconsistent or poor visibility within *hybrid* system.

② **Consolidated Monitoring Solution:** Multi-Cloud environment consists of multiple monitoring and management solutions for each individual cloud. For instance, if a multi-cloud comprises of AWS, Azure and Google Cloud Platform then, each individual environment has its own dashboard and set of APIs for monitoring and management, but there is no provision for centralized monitoring of all the three individual clouds. Thus, one possible solution is to implement a consistent data store to gather, analyze and operate on multiple metrics and events that are generated from countless containers and applications running on them. As each container and service writes its event or metrics data to a consistent API, a consolidated monitoring solution can be achieved.

E. FUTURE TREND

OS-virtualization is continuously gaining importance, however, it frequently adds layers of *indirection* and *abstraction* beneath the application code to an already densely-layered software stack that includes supports for legacy physical protocols; irrelevant optimization (like: disk elevator algorithms on SSD drives); backward compatible interfaces; user-space processes and threads; and managed-code run times. The situation will continue to become extremely critical in coming years if the inclusion of extra layers maintains current pace (due to the scattered presence of logic across different software components, written in different languages), as future programmers need to dig through thousands of layers for debugging even simplest applications.

① **Unikernels:** MirageOS (unikernel stack built in OCaml) aims to unify these diverse interfaces-both kernel and user application spaces-into a single high-level language framework. MirageOS allows all software layers of a virtual

appliance to be compiled within the same high level language framework instead of dynamically assembling them on every boot. *Mirage* (contributes a suit of type-safe protocol *libs*) dubbed Unikernel and restructures all kernel and user-space code into more flexible, secure and reusable modular components following legacy *libOS* style [67]. Unikernels are light-weight (size of few MBs), compact, single address space, memory safe, single-purpose appliances (VMS) that are compile-time specialized into standalone kernels, and sealed against modification when deployed to a cloud platform. In return they offer significant reduction in image sizes, improved efficiency and security, and should reduce operational costs [67], [118]. For example, no OS facilities for device drivers are included in the final production image of the application that does not require persistent disk access. Network latency remains a prominent problem for all cloud service. One of the ways to handle it is moving computations out of remote datacenters by rapidly instantiating local services near the user. It requires an embedded cloud platform on which to deploy multiple applications securely and quickly. In an Unikernel *edge* computing application, *Jitsu* satisfies the demands of secure multi-tenant isolation on resource-constrained embedded ARM devices [119]. Using fast shared memory channels, *Jitsu* provides a directory service that launches Unikernels in response to network traffic and masks boot latency. In other IoT edge offloading architecture, *Cozzolino et al.* [120] have utilized *MirageOS* unikernels to isolate and embed application logic in concise *Xen*-bootable images. Motifs behind adopting Unikernel in [120] are respectively the single purpose task without offloading full application and better security. Other application of Unikernels in Content Delivery Network (CDN) is found in [121]. Unikernels provide better security and isolation than their light-weight counterpart containers and in many cases they efficiently provide essential *features* like extremely small disk- and memory footprints, microscopic spin up/down times, efficient asynchronous I/O, OS-library that includes only what service needs [122]. However, upgraded security and isolation along with further reduction in booting time and complexity of Unikernels can be achieved using Unikernel monitors [123], where Unikernel is bundled with a tiny, specialized monitor that only contains required interfaces and implementation resources for the Unikernel. Few important approaches for constructing Unikernel are ❶ *ClickOS* (boots under 30ms) and *LING* (boots under 100ms) with speed as focus; ❷ *MirageOS* and *HalVM* with focus as safety and security; ❸ *Rump Kernels* and *OS^v* with focus as compatibility with legacy software; ❹ *Unikernel Linux* for handling massive range of Linux conditions [124]; ❺ *Micropython* unikernel has image size 1MB and needs 8MB of memory to run, whereas *Xen* hypervisor based VM images might often lie between 1.5 to 3 GBs.

❷ **Microkernels:** History of microkernels started around 1980's because of storage limitation of early computers. Microkernels are minimalistic kernels that include primitives

such as basic system IPC protocols (for providing mechanism for kernel-controlled change of execution flow, data transfer and resource delegation between potentially disturbing multi-party protection domains), memory management functionality necessary to run an OS, whereas every other utility is loaded in user mode (during user logging in). In other way, here the prime goal is to minimize kernel code and outsource various utilities to user space. In this way it reduces the code complexity, disk and memory footprints but at the cost of reduction in essential security mechanisms within kernel space. On contrary, VMMs closely resemble processor hardware and offer a rich variety of primitives, each of which require dedicated set of security mechanisms, resources and kernel code [125]. General VMM primitives are synchronous switch of protection domains from guest users to guest kernel and vice versa; asynchronous communication channels across domains; resource allocation within VMs; resource allocation per VM; resource reallocation; page fault and exception handling; cross-domain asynchronous event modification; hardware interrupt modification; and a set of common devices. Microkernels aim to provide minimal layer of privileged software, while VMMs rely on replicating and multiplexing hardware resources [126]. Modern VMMs are increasingly becoming more microkernel-like by outsourcing drivers to user-space for providing better legacy code support and reducing attack surface. On contrary, microkernels also support virtualization for providing better support for legacy software [126]. Unlike unikernels, microkernels support multiple applications which are protected from each other through hardware and software firewalls [127]. Some popular microkernels are *F9* (constructing power-efficient and security-sensitive real-time and embedded systems for ARM Cortex M series); *Genode* (toolkit for building highly secured special purpose OS); *HelenOS*; *M3* (aiming to support arbitrary cores), *SeL4* (high assurance and high performance microkernels), etc.

Although, unikernel and microkernel both provide lightweight functionality, microkernels can be a bit slower than its counterpart due to the messages passing mechanism between user and kernel-spaces. Moreover, microkernel has relatively larger memory footprints; they may suffer from performance degradation due to message passing between user and kernel spaces; and overall process management can be more complex than unikernels.

❸ **LightVMs:** In general, lightweight VMs are unable to ensure good performance as virtualization control plane becomes the performance bottleneck. *LightVMs* [128], [129], using *paravirtualization* (providing VM-like isolation & security and container-like performance) can also provide solution for the challenges such as: improving isolation & security, improving booting time and reducing disk and memory footprints. In *LightVMs* redesign *Xen*'s centralized control plane to a distributed one and significantly minimizes the interaction with hypervisor. *LightVMs* are compatible with *Linux fork/exec* and often found showing faster booting time than *Docker*. *LightVM* can pack and run

large numbers of lightweight guests on a modest hardware platform. LightVMs in combination with application-level virtualization can achieve resource adaptation in middleware especially for soft real-time applications using both resource managers (multiplex or map high-level resources on top of low-level resources) and resource schedulers (manages processing resources such as threads or virtual processors) operating at higher (virtual memory, network connection, team of threads, etc.) and lower-levels (physical resources like CPU, memory and network) [130].

④ **Other Approaches:** Kata containers, Clear containers, Photon OS and Hyper are well accepted lightweight virtualization mechanisms by cloud computing communities and industries [131]. Kata container initiative is an open source project managed by OpenStack Foundation that combines Intel's Clear container and HyperV's runV to run one container per compatible with OCI specifications similar to Docker [132]. Kata containers being currently in formation stages can be a next-gen virtualization technique as it offers speed of containers and security of VMs providing compatibility with major cloud platforms like with major cloud platforms like OpenStack, Google Cloud Platform and Azure. Immutable server is considered to be a deployment model that carefully endorses any application update, security patch, configuration change done to specific production system and, every time a new image is constructed, pushed and cycled to production, if, in case, any of the layers requires modifications. The advantage of this approach includes higher code confidence during production.

IX. PERFORMANCE COMPARISONS OF HYPERVISOR AND LIGHTWEIGHT VIRTUALIZATION

This section presents a performance comparison (in terms of throughput, bandwidth and network bandwidth) of container with that of native system, unikernels and VMs, while *one or more* system resources are fully utilized. For that, we used *five* standard benchmarks called RandomAccess (measures Giga Updates/sec or GUPS); SysBench (CPU performance) Linpack (numerically intensive test for measuring system's floating point performance); Iperf (for checking network bandwidth) and STREAM (measures sustained bandwidth of entire cache hierarchy). Experimental verification were done using HP-Blade server with 64 bit Ubuntu16:04, kernel 4.4, Intel Xeon(R) CPU E5-2690 2.6 GHz (64 cores), RAM 128GB, 5.4TB SSD. Base image for Docker (v18.06.1-ce) and LXC/LXD (v2.0.11) is Ubuntu16:04. We choose tiny OS^v and Rumprun VMs (size around 5 MBs) on top of Xen and kvm as unikernel representatives with extremely fast booting time and with negligible delay. Ubuntu16:04 within Xen is used as VM representative for the current experiment.

This section extends the work of Felter *et al.* [133] by extending the performance comparisons among containers (LXC/LXD, Docker, rkt), VMs (using Ubuntu16:04 VMs on top of Xen, kvm), and unikernels (OS^v, Rumprun

using Rump kernel contains a heavily-reduced version of NetBSD [124]) using standard benchmarks called RandomAccess (random memory access), SysBench (CPU performance), LINPACK (floating point rate of execution), Iperf (network bandwidth), and STREAM (measuring memory bandwidth). We have also identified primary performance impact of several instances of different virtualization appliances. During above experiments, power management was disabled using performance cpufreq governor. Docker and LXC/LXD containers were not restricted by cgroups so they could consume full system resources. They are configured with 64 vCPUs and 32 GB RAM for these experiments. We use microbenchmarks to measure CPU, memory, network, and storage overheads. Next, we fix strategies to make our experiments *robust*.

A. EXPERIMENT STRATEGY (ES)

We conducted experiments following *two* ways below:

① **ES1:** Monitoring performance of single instance (utilizing available system resources) of each of the virtual appliances at a particular time slice by running individual benchmark.

② **ES2:** Running individual benchmark simultaneously in multiple instances of each of the virtual appliances. This experiment is to show if all available channelized to a specific instance, still it can offer near native performance. For this experiment, we took 100 instances for each item.

We have conducted both ES1 and ES2 experiments 100 times and results are presented next.

B. RandomAccess

It is used to calculate *peak* capabilities of memory systems/sub-systems (like GUPS rating of a distributed memory multiprocessor, an SMP node, and that of a single processor), where GUPS is random memory access performance measurement that profiles system memory architecture. It initializes a large section of memory as its working set that is larger than caches or TLB.

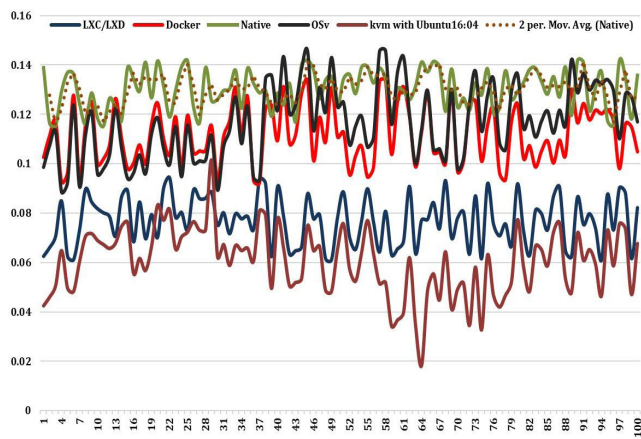
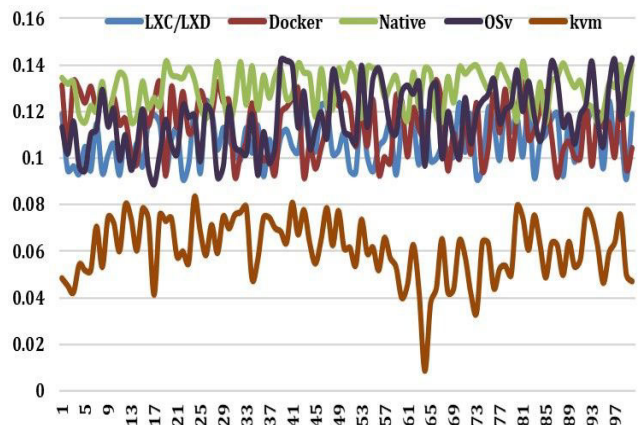
Random 8-byte words in this memory section are read, modified and dumped back. Random locations are produced using a linear feedback shift register requiring no memory operations. Hence, there is no dependency between successive operations permitting multiple independent operations to be in flight through the system.

RandomAccess typifies the behavior of workloads with large working sets and minimal computation such as those with in-memory hash tables and in-memory databases. RandomAccess uses large pages to reduce TLB miss overhead. Due to its random memory access pattern and a working set that is larger than the TLB reach, RandomAccess significantly exercises the hardware page table walker that handles TLB misses.

Fig. 2 shows Docker and OS^v have achieved near-native GUPS performance, whereas LXC/LXD staged with next best GUPS performance. kvm with Ubuntu16:04 showed the weakest GUPS performance for ES1 experimental strategies.

TABLE 4. Unikernels vs. Microkernels vs. Monolithic Kernels.

	Unikernels	Microkernels	Monolithic Kernels
① Address Space	Single; mainly single-purpose appliance.	Kernel & User.	Kernel & User.
② Booting Time	Extremely fast startup (boot around 30-100ms).	Relatively slow.	Slowest, as it has to load various functionality during boot-up.
③ No. of apps	Single.	Multiple.	Multiple.
④ OS footprint	Extremely small.	Minimalistic OS with reduced functionality.	Largest footprint.
⑤ Operating Mode	Privileged only.	Privileged.	User & Privileged.
⑥ Bare-metal Compatibility	Provides support.	Provides support.	Provides support.
⑦ Security	★ In unikernel, attack surface is relatively small (only a few thousands lines of code) as it includes minimal functionality required for running an application. ★ There is no shell and issuing <code>exec()</code> for a new process is restricted. Hence, exploits have hard time to get permanent foothold in the system. ★ Hence, can provide good security mechanism with limitations.	★ It also has limited footprint and hence, less attack surface. Might provide good security isolation.	★ Attack surface is very large (a million lines of code) as it provides all possible supports that an OS should provide. ★ Shells are present and issuing <code>exec()</code> is not restricted. Various isolation and security addons can be added to increase security.
⑧ Examples	ClickOS, HalVM, MirageOS, Rump Kernel.	CMU Mach, AmigaOS, MINIX3, etc.	Linux, Solaris, Windows, etc.

**FIGURE 2. GUPS patterns for 100 RandomAccess benchmark testing for Docker, OS^v, LXC/LXD, kvm with Ubuntu16:04 image, and host for ES1 strategy. Here, x-axis represents experiment repetition, whilst y-axis depicts GUPS. We found, Docker and OS^v show near real-time GUPS performance, whereas kvm with Ubuntu16:04 shows the worst performance among all.****FIGURE 3. GUPS patterns for 100 RandomAccess benchmark testing for Docker, OS^v, LXC/LXD, kvm with Ubuntu16:04 image, and Native system for ES2 strategy (with 50 similar container and VM instances). It shows Docker, OS^v and LXC/LXD outperform kvm.**

Results are slightly different for ES2 strategy, where Docker, LXC/LXD and OS^v outperform kvm (see Fig. 3 and Table 5). Though kvm is found offering relatively weaker

GUPS performance in the pictorial representations, but in reality all virtualization appliances impose negligible overhead on memory usage.

Another experiment relating container performance focusing GUPS fluctuations with variable *global* table (vector) size and *chunk* (the number of updates each process does before communicating) size simultaneously. Current experiment is motivated from the experiments performed in <http://icl.cs.utk.edu/projectsfiles/hpcc/RandomAccess/>, however, they have not considered variable vector and *chunk* sizes simultaneously. For this experiment, initially, we have taken *global* table ($\mathbb{T}[\cdot]$) size as 2^n containing 64-bit integer in each cell and $\mathbb{A}[\cdot]$ be $2^{(n+2)}$ long stream of 64-bit integers generated by slightly changed primitive polynomial ($x^{64} + x^{62} + x^{60}$) over GF(2). Next, for each $\mathbb{A}[i]$ set $\mathbb{T}[\mathbb{A}[i] \langle 62, 64 - n \rangle] = \mathbb{T}[\mathbb{A}[i] \langle 62, 64 - n \rangle] + \mathbb{A}[i]$, where “+” in GF(2) represents “ $\oplus$ ” (XOR operation) and $\mathbb{A}[i] \langle 62, 64 - n \rangle$ is the bit sequence within $\mathbb{A}[i]$. Here, n is the largest power of 2, which is less than or equal to the half of main memory size. In short, we have to perform $\oplus$ between a specific 64 bit integer $\mathbb{T}[i]$ and another integer $\mathbb{A}[i]$ taken from the stream and will store the result back to the location $\mathbb{T}[i]$ and have to monitor corresponding GUPS fluctuations by changing the *vector*, *chunk* sizes simultaneously (see Fig. 4 and Fig. 5). GUPS performance decreases for smaller *chunk* sizes (see Fig. 5).

With our detailed experiments with RandomAccess benchmark, it can be concluded that container instances (Docker and LXC/LXD, in the current study) can also provide near real-time performances and hence, will be helpful for modern cloud industries from random access performance measurement point of view. However, for further investigations, we also worked on other benchmarks in *subsequent* subsections.

C. CPU PERFORMANCE-SysBench

We used SysBench benchmark’s CPU [134] test for checking CPU performance for native and other virtual appliances (Docker, OS^v, RumpRun, rkt, LXC/LXD, kvm, Xen) using a maximum prime number, which the test run

TABLE 5. Performance evaluation of different lightweight virtual appliances and VMs using popular benchmarks such as STREAM and LINPACK. The experiments were conducted following both $\mathcal{ES}1$ (using single instance of a virtual appliance) and $\mathcal{ES}2$ (employing multiple instances of a specific virtual appliance, columns are highlighted blue) strategies. Detailed analysis of the experimental results are presented in Section IX-E and Section IX-F. Here, “No.” implies the number of instance(s) of a specific virtual appliance running during an experiment. x-axis: number of virtual appliances running on the underlying hardware, and y-axis: data transfer rate (MB/sec).

STREAM (MB/sec) (values are neglected after decimal)																								
COPY ($a[i] := b[i]$) (avg. of 10 runs of each kernel)										SCALE ($a[i] := p * b[i]$) (avg. of 10 runs of each kernel)														
No.	Docker	rkt	LXC	OSv	Rumprun	Xen PV	Xen HVM	kvm+ VirtIO	kvm- VirtIO	No.	Docker	rkt	LXC	OSv	Rumprun	Xen PV	Xen HVM	kvm+ VirtIO	kvm- VirtIO					
1	9057	9017	9011	8065	9013	9048	8996	9016	9032	1	7011	7015	7031	6578	7015	6981	6923	6978	7098					
2	9012	9023	9037	8161	9017	9028	9001	8950	9056	2	7041	7071	7089	6521	7081	6945	6972	6980	6925					
4	9069	9003	9019	8018	9011	9025	8650	9039	9049	4	7037	7052	7036	6532	7056	6982	6981	6957	7036					
6	9072	9051	9071	8051	9081	9075	9001	9047	9071	6	7089	7093	7091	6571	7047	6783	6955	6939	6898					
8	9041	9061	9082	8075	9026	9062	8410	8940	9025	8	7096	7055	7089	6592	7988	6987	6987	6972	6931					
10	9001	9014	9034	8081	9032	8990	8231	9001	9052	10	7038	7034	7075	6588	7972	6995	6919	6986	7009					
ADD ($a[i] := b[i] + c[i]$) (avg. 10 runs of each kernel)										TRIAD ($a[i] := b[i] + p * c[i]$) (avg. 10 runs of each kernel)														
No.	Docker	rkt	LXC	OSv	Rumprun	Xen PV	Xen HVM	kvm+ VirtIO	kvm- VirtIO	No.	Docker	rkt	LXC	OSv	Rumprun	Xen PV	Xen HVM	kvm+ VirtIO	kvm- VirtIO					
1	8798	9005	9098	7564	8823	8789	9011	8876	9084	1	6579	6709	6609	6089	6459	6609	6678	6709	6789					
2	8761	9017	9077	7568	8814	8811	8901	8971	9085	2	6509	6610	6681	6009	6501	6610	6681	6784	6631					
4	8752	8969	9100	7588	8873	8834	8972	8909	8979	4	6620	6501	6657	6072	6596	6705	6649	6736	6701					
6	8799	8992	9042	7683	8891	8879	8905	8861	8807	6	6645	6671	6671	6037	6541	6694	6692	6785	6706					
8	8932	9004	9072	7611	8864	8841	8897	8834	8809	8	6539	6781	6603	6059	6498	6645	6667	6745	6788					
10	9006	8964	8997	7631	8817	8799	8965	8989	8901	10	6647	6602	6701	6109	6507	6649	6671	6761	6698					
LINPACK (MFLOPS) (avg. LINPACK (MFLOPS) of each kernel run for 100 times)																								
Native										Docker														
LXC/LXD																								
					$\mathcal{E}S1$					$\mathcal{E}S2$					$\mathcal{E}S1$					$\mathcal{E}S2$				
					510					515					482					485				

related rows are highlighted green), however, containers can also offer good performance. Unikernels are very promising, probably, by combining the best of VMs and containers for providing lightweight and secure virtualization but still need research for providing stable and steady performance throughout. Same conclusion can be drawn for containers as they show fluctuations in execution times between $\mathcal{ES}1$ and $\mathcal{ES}2$ strategies.

D. NETWORK BANDWIDTH-Iperf

We used Iperf2 [135], network benchmarking tool for measuring and comparing network bandwidth of containers (Docker, rkt), unikernels (OS^v, Rumpun), and VMs (kvm). In each case, we have tested with two instances (one acting as server and the other as client), where containers use default bridges, whereas unikernels and VMs on hypervisor were bridged using tap interface. Performance of Docker, rkt, OS^v were found more promising than that of kvm. However, kvm performance can be improved by replacing tap bridge with user space virtual switches like OSV/DPDK and VOSYSwitch.

E. MEMORY BANDWIDTH-STREAM

STREAM benchmark [136] stresses memory subsystem in a consistent fashion, permitting hardware pre-fetchers to bring in data from memory before they are used computation. Performance is dominated by system memory bandwidth, where created working set is significantly larger than cache size. Main performance determinants are main memory bandwidth and cost of handling TLB misses (to a lesser extent), which can be handled by using pages of large size. Memory access pattern is regular and hardware pre-fetchers typically latch on to the access pattern and pre-fetch data before it is needed. Performance is therefore gated by memory bandwidth and not latency. This benchmark has four operating modes: COPY, SCALE, ADD and TRIAD that are described in Table 5 with detailed results during current experimentation. Among these four modes, ADD, SCALE and TRIAD are heavily relied on CPU for arithmetic computations before writing the data to memory. On contrary, COPY measures transfer rate without doing any further arithmetic operation before writing data to memory. For this experiment, STREAM benchmark was built without “-fopenmp” flag because of the lack of OpenMP lib support available in OS^v. We also used default single thread for this experimentation with different virtual instances of containers (Docker, rkt), VMs (different Xen and kvm configurations) and unikernels (OS^v and Rumpun).

Different benchmark parameters during experiments (both $\mathcal{ES}1$ and $\mathcal{ES}2$) are Array Size ($a[.]$, $b[.]$, $c[.]$) as 1×10^7 elements, where each element is of length 8-bytes with Offset of size 0 (element); Memory per array = 76.3 MiB; Total required memory = 228.9 MiB. We have varied the number of instances of virtual appliances between 1-10 (column highlighted blue in Table 5) and each instance was run for 10 times. Experimental results point to consistently near real

time performance by Docker, rkt, LXC/LXD, Rumpun, and different configurations of kvm and Xen (see Table 5). An interesting point from Table 5 is that kvm with VirtIO performs slightly inferior than kvm without VirtIO during COPY operation (columns highlighted in green and yellow respectively in Table 5). VirtIO drivers are responsible for providing more paravirtualized methods of accessing disk blocks and network devices and hence, in its presence kvm is expected to perform better than that without the assistance of paravirtualized interface. This maybe, due to the presence of different disk caching modes (off, write through, write back) within kvm, each of which perform differently under different operating conditions. Other reason could be, kvm uses copy-on-write disk files, which might affect virtual disk performance on write operation than Xen raw image files. Detailed visualization of experimental results pictorially shown in Fig. 6.

F. FLOATING POINT RATE OF EXECUTION-Linpack

Next, we have to see CPU performance for both system and containers (Docker and LXC/LXD in the current case) using Linpack benchmarking [137], [138]. LINPACK solves a dense system of linear equations whose matrices are general, banded, *symmetric* indefinite, *symmetric positive definite*, *triangular*, and *tridiagonal* square. In addition, the package computes the QR and singular value decompositions of rectangular matrices and applies them to least-squares problems. LINPACK uses column-oriented algorithms to increase efficiency by preserving locality of reference [138]. Most of compute operations use double-precision floating point multiplication of a scalar with a vector and adding results to another vector (see Table 5). This benchmark is typically based on a linear algebra library that is heavily optimized for specific machine architecture at hand.

We use an optimized LINPACK binary (v11.1.2.005) based on Intel Math Kernel Library (MKL) with a problem size of 500×500 . Intel MKL is highly adaptive and optimizes itself based on both the available floating point resources (e.g., what form of multimedia operations are available), as well as system cache topology.

G. SUMMARY OF EXPERIMENTAL OUTCOMES & DISCUSSIONS

Modern virtual appliances (both heavier hypervisors and lightweight tools) have gained significant performance maturity from the research work of virtualization, hardware and software optimization, started around half a century ago [133]. Despite remarkable improvements, containers and unikernels are found outperforming their VM counterparts in most of the experiments. For example, random memory access performance of Docker, LXC and OS^v. (unikernel) is found better than that of kvm. On contrary, Docker fails to provide stable CPU performance (using SysBench) with increasing number of Docker instances running on the same underlying hardware ($\mathcal{ES}2$). Unikernels (OS^v. and Rumpun) show worst performance in this case and it further

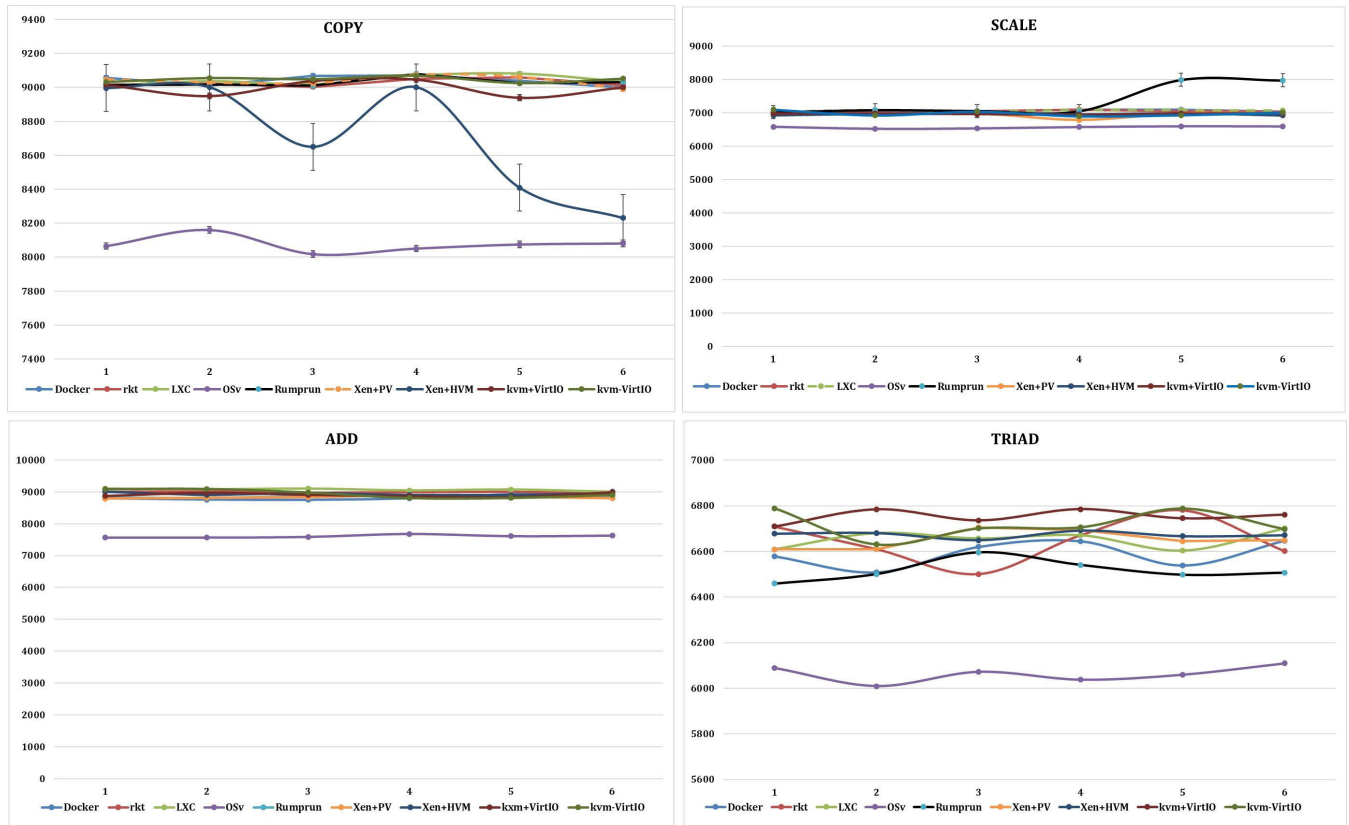


FIGURE 6. Performance analysis using STREAM benchmark: for COPY, the performance of Xen+HVM looked relatively unstable and inferior than other virtual appliances only expect OS^v. Other appliances show stable performance and Rumpun emerged as best performer for COPY operation. For SCALE, Rumpun offered the best performance and it became better with increasing number of running Rumpun instances in the underlying hardware, whereas OS^v was the consistent worst performer. Other appliances showed stable and near real-time performance. All virtual appliances offered stable and near real-time performance for ADD operation, though OS^v's performance was slightly inferior than others. For TRIAD, all appliances showed fluctuations in their performances.

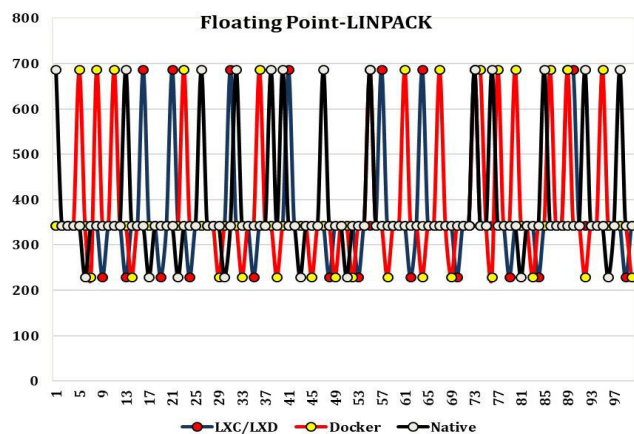


FIGURE 7. CPU usage visualizations using Docker, LXC/LXD and host.

deteriorates with increasing number of running collocated instances. Docker, rkt, and OS^v offer better network bandwidth using Iperf benchmark. For COPY, SCALE, ADD and TRIAD OS^v shows the worst performance, whereas containers and Rumpun show best performance for COPY,

SCALE and ADD; VMs and containers show stable performance throughout and for Triad VMs show the best performance. In short, unikernel performance fluctuates often, whereas that of the VMs is found most stable among all. Containerization provides near real-time, good resource utilization as it harnesses the virtues of virtualization for aggregating more applications on one system at the same time avoiding additional overheads (in terms of bigger memory footprints, slower spin up/down and deployment) cast by VMs. However, LightVMs (recent development in VM research) are able to neutralize previous VM weaknesses and additionally provide strong isolation.

Unikernels, on the other hand, offer promising features such as significantly reduced memory footprint, fast booting, VM-like security, efficient resource utilization, I/O performance and many more. Unlike containers, unikernels do not require general purpose OS like Linux, instead single purpose OS. Unikernel groups OS functionality necessary to run a specific application along with the application and allows the group to act and live independently. Hence, each unikernel has its own kernel, whereas the containers on a host have to share a kernel (and have root access too)

that may often found as the reason behind falling whole containerized data center to adversarial control through a single point of failure. Unikernels offer important advantages for some specific cases like IoT, where processing power and storage facilities are relatively scarce. Unikernels operate only in privileged mode and cannot be logged in remotely, which significantly eliminates the number of Linux system attack vectors like shell code exploits. Well-known vulnerabilities and ransomware from recent past like Equifax and SamSam respectively, were failed to hurt unikernel systems as these styles of attacks were foreign to the unikernel environment. However, unikernels are still lagged behind research in many key aspects including process containment, microprocessor stability, virtualization [124].

Hence, to continue with a steady growth in industrial applications, containerization has to provide advantages other than steady state performance, faster booting, lightweight virtualization and must have to focus on the following issues as the combination of convenience, faster and easier deployment, elasticity (cross platform support and availability of capable tools), and performance is believed to be significantly compelling in the near future:

① **Lack of Cross-Platform Supports:** Different containers do not support other platforms. For example, OpenShift only works with kubernetes orchestrator.

② **Lack of Persistent Storage Supports:** All data inside containers permanently disappear upon shutting down the specific container instance unless they are not saved in some other storage volumes. There are few existing ways to save data persistently in Docker like Docker Data Volumes that yet needs certain level of maturity and has to be implemented in more seamless way.

③ **Not fit for all Services:** Applications, implemented as microservices are the better fit for containerizations. For other services, supporting monolithic architecture, containerization only offers simplified delivery mechanism by providing easy packaging techniques.

④ **Speed Inferior to Bare-Metal:** Containers are more efficient in resource utilization than VMs but are still subject to delay because of overlay networking, interfacing between host and containers, etc.

⑤ **Weaker Isolation:** Container isolation mechanism is much weaker than that of VMs and unikernels. This is due to the kernel sharing that may lead to collapse of entire containerized environment through a single exploitable channel (single point of failure). This situation can be handled by running containers in a VM as, even though, the containers within VM are compromised, the vulnerability will not extend outside the VM, which limits the scope of potential damage.

⑥ **Handling Dependencies:** Containers have to handle multiple dependencies as they do not contain full general purpose OS. In contrast VMs are more self-contained.

⑦ **Lack of Tools and Supports:** Containerizations still need capable tools for monitoring and managing containers in real-time. However, with new tools like Kubernetes container management becomes bit easier.

⑧ **Container Sprawl:** Container life-cycle management remains critical as containers can be spun up and duplicated at remarkable rate. This could be considered as an important advantage if the unwanted container instances are deleted in regular intervals. However, this could create serious scaling issues and might lead to impose significant cloud computing costs if unwanted instances are not deleted regularly.

⑨ **Container Portability Limitations:** Porting containers from one family of OSs (say Linux) to another family (say Windows) is more complex than the porting of VMs. Additionally, a containerized Linux application needs Linux host, which is not necessary for VMs. Hence, containers are less portable in this context. Also, it is extremely hard to get container support for macOS as traditional containerization mainly supports Linux and Windows platforms. Hence, there is no native way available to deploy Dockerized applications on macOS. However, this problem can be solved using a relatively costly (in terms of higher resource utilization and complex management) technique of employing Linux-based VM as middleman between macOS and Dockerized application. However, in recent years, a new set of tools for Docker called, “LinuxKit” allows developers to create Linux subsystem within containers that provides a tiny Linux-based OS, which runs inside a container and helps Docker container to run on different types of OS. This facility is not available for other container technologies. LinuxKit is only applicable for Linux-based applications in Docker platform and till date, no similar tool exists for Windows-based applications.

X. CONCLUSION

The present study talks about several promising aspects of containerizations with detailed technicalities involved in their successful industrial implementation, security & isolation, smooth management and orchestration. In short, containerization provides many promising features like super lightweight, faster spin-up/down, efficient energy and resource utilization, impressive workload distribution capabilities, achieving server consolidation, and many more, but at the same time it has few major problems such as weaker isolation, higher chance of container sprawl, lack of capable tools for container orchestration and cross-platform supports and container portability limitations. On the other hand, VMs provide strong isolation, it has a reach set of available tools to provide cross-platform supports and management, offer better portability than containers though bigger memory footprints and slow booting remain their major issues. Unikernels provide VM-like isolation with significantly small footprints along with extremely fast booting. However, lack of efficient management and cross-platform supportive tools, unavailability of persistent storage facility, microprocessor stability and process containment remain their major problems.

However, with the recent technological developments in both VMs and lightweight virtualizations, the increasing industrial acceptance of containerization will soon be challenged as the need of the day has shifted to the advantages

other than steady state performance, faster booting, lightweight virtualization as the combination of convenience, faster and steady deployment, elasticity and performance is strongly believed to be extremely compelling in foreseeable future. Hence, to maintain its wide-spread industrial acceptance, containerization must have to focus on the following important issues like: ❶ development of tools and techniques to make container isolation stronger; ❷ development of tools for efficient container management and orchestration (although `kubernetes` is already emerged to be an efficient one); ❸ development of tools for improving portability and cross-platform support; ❹ development of tools to provide persistent storage facilities for hosted applications.

ACKNOWLEDGMENT

The authors acknowledge supports by Software Defined Communications Research/Testbed Lab, Dept. of Computer Science & Engineering @ SRM Institute of Science & Technology, India and Computer & Information Science (CIS) Dept. of Universiti Teknologi Petronas (UTP), Malaysia. The authors are thankful to the Editor, Associate Editor and the unknown Reviewers for their valuable suggestions to the major improvement of our paper.

REFERENCES

- [1] D. Bernstein, "Containers and cloud: From LXC to Docker to kubernetes," *IEEE Cloud Comput.*, vol. 1, no. 3, pp. 81–84, Sep. 2014.
- [2] M. Xu, W. Tian, and R. Buyya, "A survey on load balancing algorithms for virtual machines placement in cloud computing," *Concurrency Comput., Pract. Exper.*, vol. 29, no. 12, 2017, Art. no. e4123.
- [3] M. J. Scheepers, "Virtualization and containerization of application infrastructure: A comparison," in *Proc. 21st Twente Student Conf. IT*, 2014, pp. 1–7.
- [4] R. Uhlig, G. Neiger, D. Rodgers, A. L. Santoni, F. C. Martins, A. V. Anderson, S. M. Bennett, A. Kagi, F. H. Leung, and L. Smith, "Intel virtualization technology," *Computer*, vol. 38, no. 5, pp. 48–56, May 2005.
- [5] M. Zhang, D. Marino, and P. Efstathopoulos, "Harbormaster: Policy enforcement for containers," in *Proc. IEEE 7th Int. Conf. Cloud Comput. Technol. Sci. (CloudCom)*, Nov./Dec. 2015, pp. 355–362.
- [6] C. Zheng and D. Thain, "Integrating containers into workflows: A case study using makeflow, work queue, and Docker," in *Proc. 8th Int. Workshop Virtualization Technol. Distrib. Comput.*, 2015, pp. 31–38.
- [7] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at Google with borg," in *Proc. 10th Eur. Conf. Comput. Syst.*, 2015, p. 18.
- [8] R. Shu, X. Gu, and W. Enck, "A study of security vulnerabilities on Docker hub," in *Proc. 7th ACM Conf. Data Appl. Secur. Privacy*, 2017, pp. 269–280.
- [9] M. G. Xavier, M. V. Neves, and C. A. F. De Rose, "A performance comparison of container-based virtualization systems for mapreduce clusters," in *Proc. 22nd Euromicro Int. Conf. Parallel, Distrib., Netw.-Based Process.*, Feb. 2014, pp. 299–306.
- [10] W. Gerlach, W. Tang, K. Keegan, T. Harrison, A. Wilke, J. Bischof, M. D'Souza, S. Devoid, D. Murphy-Olson, N. Desai, and F. Meyer, "Skyport: Container-based execution environment management for multi-cloud scientific workflows," in *Proc. 5th Int. Workshop Data-Intensive Comput. Clouds (DataCloud)*, Nov. 2014, pp. 25–32.
- [11] A. Slominski, V. Muthusamy, and R. Khalaf, "Building a multi-tenant cloud service from legacy code with Docker containers," in *Proc. IEEE Int. Conf. Cloud Eng. (IC2E)*, Mar. 2015, pp. 394–396.
- [12] K.-H. Kim, J. W. Lee, M. Ben-Ami, H. Nam, J. Janak, and H. Schulzrinne, "Flexible network address mapping for container-based clouds," in *Proc. 1st IEEE Conf. Netw. Softw. (NetSoft)*, Apr. 2015, pp. 1–5.
- [13] Y. Yu, H. Zou, W. Tang, L. Liu, and F. Teng, "Flex tuner: A flexible container-based tuning system for cloud applications," in *Proc. IEEE Int. Conf. Cloud Eng.*, Mar. 2015, pp. 145–154.
- [14] P. C. V. Varma, K. V. K. Chakravarthy, V. V. Kumari, and S. V. Raju, "Analysis of a network IO bottleneck in big data environments based on Docker containers," *Big Data Res.*, vol. 3, pp. 24–28, Apr. 2016.
- [15] S. Soltész, H. Pözl, M. E. Fiuczynski, A. Bavier, and L. Peterson, "Container-based operating system virtualization: A scalable, high-performance alternative to hypervisors," in *Proc. ACM SIGOPS Oper. Syst. Rev.*, vol. 41, 2007, pp. 275–287.
- [16] T. Combe, A. Martin, and R. Di Pietro, "To Docker or not to Docker: A security perspective," *IEEE Cloud Comput.*, vol. 3, no. 5, pp. 54–62, Sep./Oct. 2016.
- [17] B. Arnold, S. A. Baset, P. Dettori, M. Kalantar, I. I. Mohamed, S. J. Nadgowda, M. Sabath, S. R. Seelam, M. Steinder, and M. Spreitzer, "Building the IBM containers cloud service," *IBM J. Res. Develop.*, vol. 60, nos. 2–3, pp. 9:1–9:12, 2016.
- [18] L. Li, T. Tang, and W. Chou, "A REST service framework for fine-grained resource management in container-based cloud," in *Proc. IEEE 8th Int. Conf. Cloud Comput. (CLOUD)*, Jun./Jul. 2015, pp. 645–652.
- [19] L. Baresi, S. Guinea, G. Quattrocchi, and D. A. Tamburri, "MicroCloud: A container-based solution for efficient resource management in the cloud," in *Proc. IEEE Int. Conf. Smart Cloud (SmartCloud)*, Nov. 2016, pp. 218–223.
- [20] R. Wu, Y. Chen, E. Blasch, B. Liu, G. Chen, and D. Shen, "A container-based elastic cloud architecture for real-time full-motion video (FMV) target tracking," in *Proc. IEEE Appl. Imag. Pattern Recognit. Workshop (AIPR)*, Oct. 2014, pp. 1–8.
- [21] M. Mohamed, S. Yangui, S. Moalla, and S. Tata, "Web service micro-container for service-based applications in cloud environments," in *Proc. IEEE 20th Int. Workshops Enabling Technol., Infrastruct. Collaborative Enterprises (WETICE)*, Jun. 2011, pp. 61–66.
- [22] I. Jimenez, C. Maltzahn, A. Moody, K. Mohror, J. Lofstead, R. Arpaci-Dusseau, and A. Arpaci-Dusseau, "The role of container technology in reproducible computer systems research," in *Proc. IEEE Int. Conf. Cloud Eng. (IC2E)*, Mar. 2015, pp. 379–385.
- [23] S. G. Sáez, V. Andrikopoulos, R. J. Sánchez, F. Leymann, and J. Wettinger, "Dynamic tailoring and cloud-based deployment of containerized service middleware," in *Proc. IEEE 8th Int. Conf. Cloud Comput. (CLOUD)*, Jun./Jul. 2015, pp. 349–356.
- [24] R. Morabito, J. Kjällman, and M. Komu, "Hypervisors vs. lightweight virtualization: A performance comparison," in *Proc. IEEE Int. Conf. Cloud Eng. (IC2E)*, Mar. 2015, pp. 386–393.
- [25] M. Amaral, J. Polo, D. Carrera, I. Mohamed, M. Unuvar, and M. Steinder, "Performance evaluation of microservices architectures using containers," in *Proc. IEEE 14th Int. Symp. Netw. Comput. Appl. (NCA)*, Sep. 2015, pp. 27–34.
- [26] A. S. Babu, M. J. Hareesh, J. P. Martin, S. Cherian, and Y. Sastri, "System performance evaluation of para virtualization, container virtualization, and full virtualization using Xen, OpenVZ, and XenServer," in *Proc. 4th Int. Conf. Adv. Comput. Commun. (ICACC)*, Aug. 2014, pp. 247–250.
- [27] E. Truyen, D. V. Landuyt, V. Reniers, A. Rafique, B. Lagaisse, and W. Joosen, "Towards a container-based architecture for multi-tenant SaaS applications," in *Proc. 15th Int. Workshop Adapt. Reflective Middleware*, 2016, pp. 1–6.
- [28] Y. Qiu, C. H. Lung, S. Ajila, and P. Srivastava, "LXC container migration in cloudlets under multipath TCP," in *Proc. Int. Comput. Softw. Appl. Conf.*, vol. 2, Jul. 2017, pp. 31–36.
- [29] X. Guan, X. Wan, B.-Y. Choi, S. Song, and J. Zhu, "Application oriented dynamic resource allocation for data centers using Docker containers," *IEEE Commun. Lett.*, vol. 21, no. 3, pp. 504–507, Mar. 2017.
- [30] M. Mattetti, A. Shulman-Peleg, Y. Allouche, A. Corradi, S. Dolev, and L. Foschini, "Securing the infrastructure and the workloads of linux containers," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, Sep. 2015, pp. 559–567.
- [31] T. Adufu, J. Choi, and Y. Kim, "Is container-based technology a winner for high performance scientific applications?" in *Proc. 17th Asia-Pacific Netw. Oper. Manage. Symp. (APNOMS)*, Aug. 2015, pp. 507–510.
- [32] H. Kang, M. Le, and S. Tao, "Container and microservice driven design for cloud infrastructure DevOps," in *Proc. IEEE Int. Conf. Cloud Eng. (IC2E)*, Apr. 2016, pp. 202–211.

- [33] M. Raho, A. Spyridakis, M. Paolino, and D. Raho, "KVM, Xen and Docker: A performance analysis for ARM based NFV and cloud computing," in *Proc. IEEE 3rd Workshop Adv. Inf., Electron. Elect. Eng. (AIEEE)*, Nov. 2015, pp. 1–8.
- [34] A. Khan, "Key characteristics of a container orchestration platform to enable a modern application," *IEEE Cloud Comput.*, vol. 4, no. 5, pp. 42–48, Sep./Oct. 2017.
- [35] T. Inagaki, Y. Ueda, and M. Ohara, "Container management as emerging workload for operating systems," in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, Sep. 2016, pp. 1–10.
- [36] U. Awada and A. Barker, "Improving resource efficiency of container-instance clusters on clouds," in *Proc. 17th IEEE/ACM Int. Symp. Cluster, Cloud Grid Comput. (CCGRID)*, May 2017, pp. 929–934.
- [37] H. Zeng, B. Wang, W. Deng, and W. Zhang, "Measurement and evaluation for Docker container networking," in *Proc. Int. Conf. Cyber-Enabled Distrib. Comput. Knowl. Discovery (CyberC)*, Oct. 2017, pp. 105–108.
- [38] J. Anderson, H. Hu, U. Agarwal, C. Lowery, H. Li, and A. Apon, "Performance considerations of network functions virtualization using containers," in *Proc. Int. Conf. Comput., Netw. Commun. (ICNC)*, Feb. 2016, pp. 1–7.
- [39] E. Bacis, S. Mutti, S. Capelli, and S. Paraboschi, "DockerPolicyModules: Mandatory access control for Docker containers," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, Sep. 2015, pp. 749–750.
- [40] E. Casalicchio, "Autonomic orchestration of containers: Problem definition and research challenges," in *Proc. VALUETOOLS*, 2016.
- [41] J. Stubbs, W. Moreira, and R. Dooley, "Distributed systems of microservices using Docker and Serfnode," in *Proc. 7th Int. Workshop Sci. Gateways (IWSG)*, Jun. 2015, pp. 34–39.
- [42] Z. Jian and L. Chen, "A defense method against Docker escape attack," in *Proc. Int. Conf. Cryptogr., Secur. Privacy*, 2017, pp. 142–146.
- [43] W. Li, A. Kanso, and A. Gherbi, "Leveraging linux containers to achieve high availability for cloud services," in *Proc. IEEE Int. Conf. Cloud Eng. (IC2E)*, Mar. 2015, pp. 76–83.
- [44] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2014, pp. 305–319.
- [45] S. Walraven, W. De Borger, B. Vanbrabant, B. Lagaisse, D. Van Landuyt, and W. Joosen, "Adaptive performance isolation middleware for multi-tenant SaaS," in *Proc. 8th Int. Conf. Utility Cloud Comput.*, Dec. 2015, pp. 112–121.
- [46] A. Rafique, D. Van Landuyt, B. Lagaisse, and W. Joosen, "Policy-driven data management middleware for multi-cloud storage in multi-tenant SaaS," in *Proc. IEEE/ACM 2nd Int. Symp. Big Data Comput. (BDC)*, Dec. 2015, pp. 78–84.
- [47] S. Walraven, D. Van Landuyt, A. Rafique, B. Lagaisse, and W. Joosen, "PaaS Hopper: Policy-driven middleware for multi-PaaS environments," *J. Internet Services Appl.*, vol. 6, no. 1, p. 2, 2015.
- [48] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, omega, and kubernetes," Google INC, Tech. Rep., Jan./Feb. 2016, pp. 70–93. [Online]. Available: <https://storage.googleapis.com/pub-tools-public-publication-data/pdf/44843.pdf>
- [49] A. Tosatto, P. Rui, and A. Attanasio, "Container-based orchestration in cloud: State of the art and challenges," in *Proc. 9th Int. Conf. Complex, Intell., Softw. Intensive Syst. (CISIS)*, Jul. 2015, pp. 70–75.
- [50] P. Sharma, L. Chaufourier, P. Shenoy, and Y. C. Tay, "Containers and virtual machines at scale: A comparative study," in *Proc. 17th Int. Middleware Conf.*, 2016, p. 1.
- [51] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Inf. Syst. Secur.*, vol. 4, pp. 224–274, Aug. 2001.
- [52] L. Baresi, S. Guinea, A. Leva, and G. Quattrocchi, "A discrete-time feedback controller for containerized cloud applications," in *Proc. 24th ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, 2016, pp. 217–228.
- [53] P. Svärd, B. Hudzia, J. Tordsson, and E. Elmroth, "Evaluation of delta compression techniques for efficient live migration of large virtual machines," *ACM SIGPLAN Notices*, vol. 46, no. 7, pp. 111–120, 2011.
- [54] T. Bui, "Analysis of Docker security," 2015, *arXiv:1501.02967*. [Online]. Available: <https://arxiv.org/abs/1501.02967>
- [55] C. Pahl, "Containerization and the PaaS cloud," *IEEE Cloud Comput.*, vol. 2, no. 3, pp. 24–31, May/Jun. 2015.
- [56] M. Nelson, B.-H. Lim, and G. Hutchins, "Fast transparent migration for virtual machines," in *Proc. USENIX Annu. Tech. Conf., Gen. Track*, 2005, pp. 391–394.
- [57] P. Svärd, B. Hudzia, S. Walsh, J. Tordsson, and E. Elmroth, "Principles and performance characteristics of algorithms for live VM migration," *ACM SIGOPS Oper. Syst. Rev.*, vol. 49, no. 1, pp. 142–155, 2015.
- [58] A. Kivity, D. Laor, G. Costa, P. Enberg, N. Har'El, D. Marti, and V. Zolotarov, "OSv—Optimizing the operating system for virtual machines," in *Proc. USENIX ATC, USENIX Annu. Tech. Conf.*, 2014, p. 61.
- [59] S. Ibrahim, H. Jin, L. Lu, L. Qi, S. Wu, and X. Shi, "Evaluating mapreduce on virtual machines: The Hadoop case," in *Proc. IEEE Int. Conf. Cloud Comput.*, Springer, 2009, pp. 519–528.
- [60] F. Zhang, G. Liu, X. Fu, and R. Yahyapour, "A survey on virtual machine migration: Challenges, techniques, and open issues," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 2, pp. 1206–1243, 2nd Quart., 2018.
- [61] M. Agiwal, A. Roy, and N. Saxena, "Next generation 5G wireless networks: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 3, pp. 1617–1655, 3rd Quart., 2016.
- [62] A. Mirkin, A. Kuznetsov, and K. Kolyshekin, "Containers checkpointing and live migration," in *Proc. Linux Symp.*, vol. 2, 2008, pp. 85–90.
- [63] S. Nadgouda, S. Suneja, N. Bila, and C. Isci, "Voyager: Complete container state migration," in *Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jun. 2017, pp. 2137–2142.
- [64] X. Gao, Z. Gu, M. Kayaalp, D. Pendarakis, and H. Wang, "Container-Leaks: Emerging security threats of information leakages in container clouds," in *Proc. 47th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, Jun. 2017, pp. 237–248.
- [65] E. W. Biederman, "Multiple instances of the global linux namespaces," in *Proc. Linux Symp.*, vol. 1, 2006, pp. 101–112.
- [66] R. Rosen, "Resource management: Linux kernel namespaces and cgroups," *Haifux*, vol. 186, May 2013. [Online]. Available: <https://sites.cs.ucsb.edu/~rich/class/old.cs290/papers/lxc-namespace.pdf>
- [67] A. Madhavapeddy and D. J. Scott, "Unikernels: The rise of the virtual library operating system," *Commun. ACM*, vol. 57, no. 1, pp. 61–69, 2014.
- [68] Accessed: May 30, 2019. [Online]. Available: <https://www.sdxcentral.com/reports/container-cloud-orchestration-devops-download-2017/>
- [69] Accessed: May 30, 2019. [Online]. Available: <https://blog.aquasec.com/container-security-in-the-enterprise-survey-part-one>
- [70] Accessed: Oct. 21, 2019. [Online]. Available: <https://www.suse.com/products/server/jeos/>
- [71] C. Anderson, "Docker [software engineering]," *IEEE Softw.*, vol. 32, no. 3, pp. 102–c3, May/Jun. 2015.
- [72] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, "Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility," *Future Generat. Comput. Syst.*, vol. 25, no. 6, pp. 599–616, 2009.
- [73] B. I. Ismail, E. M. Goortani, M. B. A. Karim, W. M. Tat, S. Setapa, J. Y. Luke, and O. H. Hoe, "Evaluation of Docker as edge computing platform," in *Proc. IEEE Conf. Open Syst. (ICOS)*, Aug. 2015, pp. 130–135.
- [74] B. D. Ligneris, "Virtualization of linux based computers: The linux-VServer project," in *Proc. 19th Int. Symp. High Perform. Comput. Syst. Appl. (HPCS)*, May 2005, pp. 340–346.
- [75] S. S. Kumaran, *Practical LXC and LXD*. New York, NY, USA: Apress, 2017.
- [76] S. Osman, D. Subhraveti, G. Su, and J. Nieh, "The design and implementation of Zap: A system for migrating computing environments," *ACM SIGOPS Oper. Syst. Rev.*, vol. 36, no. SI, pp. 361–376, 2002.
- [77] C. Dall, J. Andrus, A. V. Hof, O. Laadan, and J. Nieh, "The design, implementation, and evaluation of cells: A virtual smartphone architecture," *ACM Trans. Comput. Syst.*, vol. 30, no. 3, p. 9, 2012.
- [78] J. Andrus, C. Dall, A. V. Hof, O. Laadan, and J. Nieh, "Cells: A virtual mobile smartphone architecture," in *Proc. 23rd ACM Symp. Oper. Syst. Princ.*, 2011, pp. 173–187.
- [79] C. Pahl, A. Brogi, J. Soldani, and P. Jamshidi, "Cloud container technologies: A state-of-the-art review," *IEEE Trans. Cloud Comput.*, vol. 7, no. 3, pp. 677–692, Jul./Sep. 2017.
- [80] V. Marmol, R. Jnagal, and T. Hockin, "Networking in containers and container clusters," *Proc. netdev 0.1*, Feb. 2015, pp. 1–4.
- [81] J. Claassen, R. Koning, and P. Grosso, "Linux containers networking: Performance and scalability of kernel modules," in *Proc. IEEE/IFIP Netw. Oper. Manage. Symp. (NOMS)*, Apr. 2016, pp. 713–717.
- [82] M. Hausenblas, *Container Networking from Docker to Kubernetes*. Sebastopol, CA, USA: O'Reilly Media, 2018.

- [83] C. Costache, O. Machidon, A. Mladin, F. Sandu, and R. Bocu, "Software-defined networking of linux containers," in *Proc. RoEduNet 8th Conf. 13th Ed., Netw. Educ. Res. Joint Event RENAM*, Sep. 2014, pp. 1–4.
- [84] Y. Li and M. Chen, "Software-defined network function virtualization: A survey," *IEEE Access*, vol. 3, pp. 2542–2553, 2015.
- [85] E. Reshetova, J. Karhunen, T. Nyman, and N. Asokan, "Security of OS-level virtualization technologies," in *Nordic Conference on Secure IT Systems*. Cham, Switzerland: Springer, 2014, pp. 77–93.
- [86] R. Pike, D. Presotto, K. Thompson, H. Trickey, and P. Winterbottom, "The use of name spaces in plan 9," in *Proc. 5th Workshop ACM SIGOPS Eur. Workshop, Models Paradigms Distrib. Syst. Struct.*, 1992, pp. 1–5.
- [87] D. J. Walsh, *Bringing New Security Features to Docker*. Opensource.com, 2014.
- [88] M. F. Bari, R. Boutaba, R. Esteves, L. Z. Granville, M. Podlesny, M. G. Rabbani, Q. Zhang, and M. F. Zhani, "Data center network virtualization: A survey," *IEEE Commun. Surveys Tuts.*, vol. 15, no. 2, pp. 909–928, 2nd Quart., 2013.
- [89] P.-H. Kamp and R. N. M. Watson, "Jails: Confining the omnipotent root," in *Proc. 2nd Int. SANE Conf.*, vol. 43, 2000, p. 116.
- [90] D. Price and A. Tucker, "Solaris zones: Operating system support for consolidating commercial workloads," in *Proc. LISA*, vol. 4, 2004, pp. 241–254.
- [91] B. Varghese, N. Wang, J. Li, and D. S. Nikolopoulos, "Edge-as-a-service: Towards distributed cloud architectures," 2017, *arXiv:1710.10090*. [Online]. Available: <https://arxiv.org/abs/1710.10090>
- [92] R. Petrolo, R. Morabito, V. Loscri, and N. Mitton, "The design of the gateway for the cloud of things," *Ann. Telecommun.*, vol. 72, nos. 1–2, pp. 31–40, 2017.
- [93] K. Jaiswal, S. Sobhanayak, A. K. Turuk, S. L. Bibhudatta, B. K. Mohanta, and D. Jena, "An Iot-cloud based smart healthcare monitoring system using container based virtual environment in edge device," in *Proc. Int. Conf. Emerg. Trends Innov. Eng. Technol. Res. (ICETIETR)*, Jul. 2018, pp. 1–7.
- [94] D. J. Dean, R. Ranchal, Y. Gu, A. Sailer, S. Khan, K. Beaty, S. Bakthavachalam, Y. Yu, Y. Ruan, and P. Bastide, "Engineering scalable, secure, multi-tenant cloud for healthcare data," in *Proc. IEEE World Congr. Services (SERVICES)*, Jun. 2017, pp. 21–29.
- [95] A. Palesandro, C. G. Guegan, M. Lacoste, and N. Bennani, "Overcoming barriers for ubiquitous user-centric healthcare services," *IEEE Cloud Comput.*, vol. 3, no. 6, pp. 64–74, Nov/Dec. 2016.
- [96] J. McPadden, T. J. S. Durant, D. R. Bunch, A. Coppi, N. Price, K. Rodgeron, C. J. Torre, Jr., W. Byron, H. P. Young, A. L. Hsiao, H. M. Krumholz, and W. L. Schulz, "A scalable data science platform for healthcare and precision medicine research," 2018, *arXiv:1808.04849*. [Online]. Available: <https://arxiv.org/abs/1808.04849>
- [97] P. Boonma, J. Natwichai, K. Khwanngern, and P. Nantawad, "DAHS: A distributed data-as-a-service framework for data analytics in healthcare," in *Proc. Int. Conf. P2P, Parallel, Grid, Cloud Internet Comput.* Cham, Switzerland: Springer, 2018, pp. 486–495.
- [98] K. Han, S. Li, S. Tang, H. Huang, S. Zhao, G. Fu, and Z. Zhu, "Application-driven end-to-end slicing: When wireless network virtualization orchestrates with NFV-based mobile edge computing," *IEEE Access*, vol. 6, pp. 26567–26577, 2018.
- [99] M. Pattaranantakul, R. He, Z. Zhang, A. Meddahi, and P. Wang, "Leveraging network functions virtualization orchestrators to achieve software-defined access control in the clouds," *IEEE Trans. Dependable Secure Comput.*, to be published.
- [100] L. Li, K. Ota, and M. Dong, "DeepNFV: A lightweight framework for intelligent edge network functions virtualization," *IEEE Netw.*, vol. 33, no. 1, pp. 136–141, Jan./Feb. 2019.
- [101] F.-H. Tseng, M.-S. Tsai, C.-W. Tseng, Y.-T. Yang, C.-C. Liu, and L.-D. Chou, "A lightweight autoscaling mechanism for fog computing in industrial applications," *IEEE Trans. Ind. Informat.*, vol. 14, no. 10, pp. 4529–4537, Oct. 2018.
- [102] A. Boubendir, E. Bertin, and N. Simoni, "On-demand dynamic network service deployment over NaaS architecture," in *Proc. IEEE/IFIP Netw. Oper. Manage. Symp. (NOMS)*, Apr. 2016, pp. 1023–1024.
- [103] R. Riggio, S. N. Khan, T. Subramanya, I. G. B. Yahia, and D. Lopez, "LightMANO: Converging NFV and SDN at the edges of the network," in *Proc. IEEE/IFIP Netw. Oper. Manage. Symp. (NOMS)*, Apr. 2018, pp. 1–9.
- [104] A. Balalaie, A. Heydarnoori, and P. Jamshidi, "Microservices architecture enables devOps: Migration to a cloud-native architecture," *IEEE Softw.*, vol. 33, no. 3, pp. 42–52, May/June. 2016.
- [105] J. Thönes, "Microservices," *IEEE Softw.*, vol. 32, no. 1, p. 116, Jan./Feb. 2015.
- [106] N. Dragoni, I. Lanese, S. T. Larsen, M. Mazzara, R. Mustafin, and L. Safina, "Microservices: How to make your application scale," in *Proc. Int. Andrei Ershov Memorial Conf. Perspect. Syst. Inform. Cham, Switzerland: Springer*, 2017, pp. 95–104.
- [107] P. Patel, M. I. Ali, and A. Sheth, "On using the intelligent edge for IoT analytics," *IEEE Intell. Syst.*, vol. 32, no. 5, pp. 64–69, Sep. 2017.
- [108] C. Dupont, R. Gialfreda, and L. Capra, "Edge computing in IoT context: Horizontal and vertical linux container migration," in *Proc. Global Internet Things Summit (GIoTS)*, Jun. 2017, pp. 1–4.
- [109] G. A. Carella, M. Pauls, T. Magedanz, M. Cilloni, P. Bellavista, and L. Foschini, "Prototyping NFV-based multi-access edge computing in 5G ready networks with open baton," in *Proc. IEEE Conf. Netw. Softw. (NetSoft)*, Jul. 2017, pp. 1–4.
- [110] T. Hall, "Masking logical unit numbers in a shared data storage system," U.S. Patent 6 763 455 B2, Jul. 13, 2004.
- [111] H. Yoon, M. Ravichandran, A. Gavrilovska, and K. Schwan, "Distributed cloud storage services with FLECS containers," *Open Cirrus Summit*, 2011, pp. 1–5.
- [112] C. Pahl and B. Lee, "Containers and clusters for edge cloud architectures—A technology review," in *Proc. 3rd Int. Conf. Future Internet Things Cloud (FiCloud)*, Aug. 2015, pp. 379–386.
- [113] C. Pahl, S. Helmer, L. Miori, J. Sanin, and B. Lee, "A container-based edge cloud PaaS architecture based on raspberry PI clusters," in *Proc. IEEE 4th Int. Conf. Future Internet Things Cloud Workshops (FiCloudW)*, Aug. 2016, pp. 117–124.
- [114] J. Liu, G. Shou, Y. Liu, Y. Hu, and Z. Guo, "Performance evaluation of integrated multi-access edge computing and fiber-wireless access networks," *IEEE Access*, vol. 6, pp. 30269–30279, 2018.
- [115] M. Selimi, L. Cerdà-Alabern, F. Freitag, L. Veiga, A. Sathiseelan, and J. Crowcroft, "A lightweight service placement approach for community network micro-clouds," *J. Grid Comput.*, vol. 17, no. 1, pp. 169–189, 2018.
- [116] M. A. AlZain, B. Soh, and E. Pardede, "A survey on data security issues in cloud computing: From single to multi-clouds," *J. Softw.*, vol. 8, no. 5, pp. 1068–1078, 2013.
- [117] M. Vukolić, "The Byzantine empire in the intercloud," *ACM Sigact News*, vol. 41, no. 3, pp. 105–111, 2010.
- [118] A. Madhavapeddy, R. Mortier, C. Rotsos, D. Scott, B. Singh, T. Gazagnaire, S. Smith, S. Hand, and J. Crowcroft, "Unikernels: Library operating systems for the cloud," *ACM SIGARCH Comput. Archit. News*, vol. 41, no. 1, pp. 461–472, 2013.
- [119] A. Madhavapeddy, T. Leonard, M. Skjegstad, T. Gazagnaire, D. Sheets, D. Scott, R. Mortier, A. Chaudhry, B. Singh, and J. Ludlam, "Jitsu: Just-in-time summoning of unikernels," in *Proc. 12th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2015, pp. 559–573.
- [120] V. Cuzzolino, A. Y. Ding, and J. Ott, "FADES: Fine-grained edge offloading with unikernels," in *Proc. Workshop Hot Topics Container Netw. Netw. Syst.*, 2017, pp. 36–41.
- [121] S. Kuenzer, A. Ivanov, F. Manco, J. Mendes, Y. Volchkov, F. Schmidt, K. Yasukata, M. Honda, and F. Huici, "Unikernels everywhere: The case for elastic CDNs," *ACM SIGPLAN Notices*, vol. 52, no. 7, pp. 15–29, 2017.
- [122] A. Bratterud, A.-A. Walla, H. Haugerud, P. E. Engelstad, and K. Begnum, "IncludeOS: A minimal, resource efficient unikernel for cloud services," in *Proc. IEEE 7th Int. Conf. Cloud Comput. Technol. Sci. (CloudCom)*, Nov./Dec. 2015, pp. 250–257.
- [123] D. Williams and R. Koller, "Unikernel monitors: Extending minimalism outside of the box," in *Proc. 8th USENIX Workshop Hot Topics Cloud Comput. (HotCloud)*, 2016, pp. 1–6.
- [124] A. Raza, P. Sohal, J. Cadden, J. Appavoo, U. Drepper, R. Jones, O. Krieger, R. Mancuso, and L. Woodman, "Unikernels: The next stage of linux's dominance," in *Proc. Workshop Hot Topics Oper. Syst.*, 2019, pp. 7–13.
- [125] G. Heiser, V. Uhlig, and J. LeVasseur, "Are virtual-machine monitors microkernels done right?" *ACM SIGOPS Oper. Syst. Rev.*, vol. 40, no. 1, pp. 95–99, 2006.
- [126] G. Heiser and B. Leslie, "The OKL4 microvisor: Convergence point of microkernels and hypervisors," in *Proc. 1st ACM Asia-Pacific Workshop Workshop Syst.*, 2010, pp. 19–24.

- [127] B. N. Bershad, C. Chambers, S. Eggers, C. Maeda, D. McNamee, P. Pardyak, S. Savage, and E. G. Sirer, "SPIN: An extensible microkernel for application-specific operating system services," in *Proc. 6th Workshop ACM SIGOPS Eur. Workshop, Matching Oper. Syst. Appl. Needs*, 1994, pp. 68–71.
- [128] F. Manco, C. Lupu, F. Schmidt, J. Mendes, S. Kuenzer, S. Sati, K. Yasukata, C. Raiciu, and F. Huici, "My VM is lighter (and safer) than your container," in *Proc. 26th Symp. Oper. Syst. Princ.*, 2017, pp. 218–233.
- [129] A. Whitaker, M. Shaw, and S. D. Gribble, "Denali: Lightweight virtual machines for distributed and networked applications," Univ. Washington, Seattle, WA, USA, Tech. Rep. 02-02-01, 2002.
- [130] H. A. Duran-Limon, M. Siller, G. S. Blair, A. Lopez, and J. F. Lomber-Landa, "Using lightweight virtual machines to achieve resource adaptation in middleware," *IET Softw.*, vol. 5, no. 2, pp. 229–237, Apr. 2011.
- [131] C. Puliafito, E. Mingozzi, C. Vallati, F. Longo, and G. Merlino, "Virtualization and migration at the network edge: An overview," in *Proc. IEEE Int. Conf. Smart Comput. (SMARTCOMP)*, Jun. 2018, pp. 368–374.
- [132] R. Jain, *Containers, Dockers, and Kubernetes*. Saint Louis, MO, USA: Washington Univ. in Saint Louis, 2018.
- [133] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, "An updated performance comparison of virtual machines and linux containers," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, Mar. 2015, pp. 171–172.
- [134] A. Kopytov, (2004). *Sysbench: A System Performance Benchmark*. [Online]. Available: <http://sysbench.sourceforge.net/>
- [135] V. Gueant, "iPerf-the TCP, UDP and SCTP network bandwidth measurement tool," Tech. Rep., 2017. [Online]. Available: <https://iPerf.com/wp-content/uploads/2014/10/sysbench-manual.pdf>
- [136] J. D. McCalpin, "A survey of memory bandwidth and machine balance in current high performance computers," in *Proc. IEEE TCCA Newslett.*, Jun. 1997, pp. 19–25.
- [137] Q. Ali, V. Kiriansky, J. Simons, and P. Zaroo, "Performance evaluation of HPC benchmarks on VMware's ESXi server," in *Proc. Eur. Conf. Parallel Process.* Berlin, Germany: Springer, 2011, pp. 213–222.
- [138] J. J. Dongarra and P. Luszczek, "Introduction to the HPCChallenge benchmark suite," Univ. Tennessee, Knoxville, TN, USA, Tech. Rep., 2004. [Online]. Available: <https://apps.dtic.mil/dtic/tr/fulltext/u2/a439315.pdf>



JUNZO WATADA is currently a Professor with the Department of CIS, UTP Malaysia, and also a Research Professor with the Research Institute of Quantitative Economics, Zhejiang Gongshang University, China. His research interests include big data analytics, soft computing, tracking systems, knowledge engineering, and management engineering. He received the Henri Coanda Medal Award from Inventico in Romania, in 2002, and the GH Asachi Medal from Universitatea Tehnica GH Asachi, IASI, Romania, in 2006.



ARUNAVA ROY received the Ph.D. degree from the Department of Applied Mathematics, Indian Institute of Technology Dhanbad, and held a postdoctoral position with the Department of Computer Science, The University of Memphis, Memphis, TN, USA. He has also worked as a Research Fellow with the Department of Industrial and Systems Engineering, National University of Singapore, and also with the Corp. Lab, Singapore University of Technology and Design. He is currently working as a Research Assistant Professor with the Department of CSE, SRM IST, Chennai, India, and also a Research Fellow with the Universiti Teknologi Petronas, Seri Iskandar, Perak, Malaysia.



RUTURAJ KADIKAR received the M.Tech. degree in communication systems from SRM IST, India. He also worked as a Senior Research Associate with the Software Defined Communication Laboratory, SRM IST, Chennai, India. He is currently working as a Software Engineer with InfraCloud Technologies, Pune.



HOANG PHAM is currently a Distinguished Professor and the former Chairman of the Department of Industrial and Systems Engineering, Rutgers University. He has been served as the Editor-in-Chief, Editor, Associate Editor, Guest Editor, and a board member of many journals. He is the Editor of the *Springer Book Series in Reliability Engineering* and the Editor of the *World Scientific Book Series on Industrial and Systems Engineering*. He is a Fellow of the IIE.



BING XU received the master's and Ph.D. degrees from Zhejiang University, China, and Waseda University, Japan, respectively. He is currently the Director of the Research Institute of Quantitative Economics, Zhejiang Gongshang University, and the Executive Vice Chairman and legal representative of the Zhejiang Quantitative Economics Association. He is the founder of the International Symposium on Innovative Management, Information and Production. Since 2010, he has organized successive annual IMIP conferences in different regions of Asia. His research interests are non-parametric statistical model, path identification, and big data applications.

...